EduGenie

Gemini-Powered Learning Assistant
Project Source Code PDF

This PDF contains the main documentation, configuration, server, and React/TypeScript source files from the uploaded EduGenie project. Dependency lock files and Git metadata are omitted for readability.

README.md

Run and deploy your AI Studio app

This contains everything you need to run your app locally.

View your app in AI Studio: https://ai.studio/apps/d6c8ae78-407d-4d79-86d2-3a60eb8efff5

Run Locally

Prerequisites: Node.js

1. Install dependencies:
`npm install`
2. Set the `GEMINI_API_KEY` in .env.local to your Gemini API key
3. Run the app:
`npm run dev`

.env.example

```
# GEMINI_API_KEY: Required for Gemini AI API calls.
# AI Studio automatically injects this at runtime from user secrets.
# Users configure this via the Secrets panel in the AI Studio UI.
GEMINI_API_KEY="MY_GEMINI_API_KEY"

# APP_URL: The URL where this applet is hosted.
# AI Studio automatically injects this at runtime with the Cloud Run service URL.
# Used for self-referential links, OAuth callbacks, and API endpoints.
APP_URL="MY_APP_URL"
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>EduGenie: Gemini Powered Learning Assistant</title>
    <meta name="description" content="AI tutor and learning assistant for school and college students featuring doubts solver, concept e
    <meta property="og:title" content="EduGenie: Gemini Powered Learning Assistant" />
    <meta property="og:description" content="AI tutor and learning assistant for school and college students featuring doubts solver, co
    <meta property="og:type" content="website" />
    <meta name="twitter:card" content="summary_large_image" />
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&family=JetBrains+Mono:wght@400;500;6
  </head>
  <body class="bg-slate-50 dark:bg-slate-950 text-slate-800 dark:text-slate-100 antialiased selection:bg-indigo-500/20 selection:text-ir
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

metadata.json

```
{
  "name": "EduGenie: Gemini Powered Learning Assistant",
  "description": "AI tutor and learning assistant for school and college students featuring doubts solver, concept explainer, notes summ
  "requestFramePermissions": [],
  "majorCapabilities": [
    "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
  ]
}
```

package.json

```
{
  "name": "react-example",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "tsx server.ts",
    "build": "vite build",
    "start": "tsx server.ts",
    "preview": "vite preview",
    "clean": "rm -rf dist server.js",
    "lint": "tsc --noEmit"
  },
  "dependencies": {
    "@google/genai": "^2.4.0",
    "@tailwindcss/vite": "^4.3.3",
    "@vitejs/plugin-react": "^6.1.1",
    "canvas-confetti": "^1.9.4",
    "dotenv": "^17.2.3",
    "express": "^4.21.2",
    "lucide-react": "^0.546.0",
    "motion": "^12.23.24",
    "react": "^19.0.1",
    "react-dom": "^19.0.1",
    "react-markdown": "^10.1.0",
    "remark-gfm": "^4.0.1",
    "vite": "^8.3.0"
  },
  "devDependencies": {
    "@types/canvas-confetti": "^1.9.0",
    "@types/express": "^4.17.21",
    "@types/node": "^22.14.0",
    "@types/react": "^19.3.0",
    "@types/react-dom": "^19.3.0",
    "autoprefixer": "^10.4.21",
    "esbuild": "^0.25.0",
    "tailwindcss": "^4.3.3",
    "tsx": "^4.21.0",
    "typescript": "^7.0.2"
  }
}
```

server.ts

```
import 'dotenv/config';
import express, { Request, Response } from 'express';
import path from 'path';
import { fileURLToPath } from 'url';
import { GoogleGenAI, Type } from '@google/genai';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const PORT = process.env.PORT ? parseInt(process.env.PORT, 10) : 3000;

// Middleware to parse JSON payloads (limit 15mb to support uploaded notes and small PDFs)
app.use(express.json({ limit: '15mb' }));

// Initialize Gemini SDK with User-Agent header as required
const apiKey = process.env.GEMINI_API_KEY || '';
const ai = new GoogleGenAI({
  apiKey,
  httpOptions: {
    headers: {
      'User-Agent': 'aistudio-build',
    },
  },
});

const MODEL_NAME = 'gemini-3.8-flash';

// Helper to format error responses nicely
function handleGeminiError(error: any, res: Response) {
  console.error('Gemini API Error:', error);
  const errorMessage = error?.message || String(error);

  // Rate limit / 429 error
  if (errorMessage.includes('429') || errorMessage.toLowerCase().includes('quota') || errorMessage.toLowerCase().includes('rate limit'))
    return res.status(429).json({
      error: 'Too many requests, please wait a moment and try again.',
      isRateLimit: true,
      retryAfter: 5,
    });
  }

  // Missing or invalid API key
  if (errorMessage.includes('API key') || errorMessage.includes('403') || errorMessage.includes('401')) {
    return res.status(401).json({
      error: 'Gemini API key is not configured or is invalid. Please check your environment configuration.',
    });
  }

  return res.status(500).json({
    error: 'An error occurred while generating the response. Please try again.',
    details: errorMessage,
  });
}

// ---------------------------------------------------------------------------
// 1. AI Tutor Chat (Streaming with SSE & Conversation Memory)
// ---------------------------------------------------------------------------
app.post('/api/chat/stream', async (req: Request, res: Response) => {
  const { messages, subject, gradeLevel } = req.body;

  if (!messages || !Array.isArray(messages) || messages.length === 0) {
    return res.status(400).json({ error: 'Messages array is required.' });
  }

  // Set SSE headers
  res.setHeader('Content-Type', 'text/event-stream');
  res.setHeader('Cache-Control', 'no-cache');
  res.setHeader('Connection', 'keep-alive');
  res.flushHeaders?.();

  const systemInstruction = `You are EduGenie, an expert, enthusiastic, and empathetic AI learning assistant and doubt solver for ${grad
Your pedagogical approach:
1. Explain step-by-step with clear, accessible language suited for students.
2. Break down complex ideas using intuitive analogies and everyday examples.
3. Structure your response cleanly using Markdown: bold important concepts, use organized bullet points or numbered lists, wrap code in
4. If the student asks for homework answers directly, guide them through the logic and method rather than simply stating the solution.
5. Always end your response with a friendly, short follow-up check question (1-2 sentences) to test their understanding or invite them t

  try {
    // Map client messages to Gemini contents format
    const contents = messages.map((m: { role: string; content: string }) => ({
      role: m.role === 'user' ? 'user' : 'model',
      parts: [{ text: m.content }],
    }));

    const responseStream = await ai.models.generateContentStream({
```

```
      model: MODEL_NAME,
      contents,
      config: {
        systemInstruction,
        temperature: 0.7,
      },
    });

    for await (const chunk of responseStream) {
      const text = chunk.text || '';
      if (text) {
        res.write(`data: ${JSON.stringify({ text })}\n\n`);
      }
    }

    res.write('data: [DONE]\n\n');
    res.end();
  } catch (error: any) {
    console.error('Chat stream error:', error);
    const errText = error?.message || 'Chat generation error';
    const is429 = errText.includes('429') || errText.toLowerCase().includes('quota');
    const userMsg = is429
      ? 'Too many requests, please wait a moment and try again.'
      : 'Something went wrong while generating the tutor response.';

    res.write(`data: ${JSON.stringify({ error: userMsg, isRateLimit: is429 })}\n\n`);
    res.end();
  }
});

// ---------------------------------------------------------------------------
// 2. Concept Explainer (Structured output with Definition, Analogy, Key Points, Real-World Example, Recap)
// ---------------------------------------------------------------------------
app.post('/api/explain', async (req: Request, res: Response) => {
  const { topic, level, subject } = req.body;

  if (!topic || typeof topic !== 'string' || !topic.trim()) {
    return res.status(400).json({ error: 'Topic is required.' });
  }

  const selectedLevel = level || 'Intermediate';

  const prompt = `Explain the concept "${topic.trim()}" at a ${selectedLevel} level${subject ? ` for ${subject}` : ''}.
Produce a comprehensive, engaging learning guide tailored for a student at the ${selectedLevel} level.`;

  try {
    const response = await ai.models.generateContent({
      model: MODEL_NAME,
      contents: prompt,
      config: {
        systemInstruction: `You are EduGenie's Concept Explainer engine. Provide a structured, engaging explanation suitable for a ${sel
        responseMimeType: 'application/json',
        responseSchema: {
          type: Type.OBJECT,
          properties: {
            topic: { type: Type.STRING },
            level: { type: Type.STRING },
            simpleDefinition: {
              type: Type.STRING,
              description: 'A 2-3 sentence definition that explains the concept clearly without unnecessary jargon.',
            },
            analogy: {
              type: Type.STRING,
              description: 'A creative, relatable everyday analogy (e.g. "Think of it like a library librarian...").',
            },
            keyPoints: {
              type: Type.ARRAY,
              items: { type: Type.STRING },
              description: '4 to 6 crucial takeaways or principles that define this concept.',
            },
            realWorldExample: {
              type: Type.STRING,
              description: 'A concrete example of how this concept is applied in modern technology, nature, daily life, or industry.',
            },
            recap: {
              type: Type.STRING,
              description: 'A memorable 1-2 sentence golden takeaway.',
            },
            quickCheck: {
              type: Type.OBJECT,
              properties: {
                question: { type: Type.STRING },
                options: {
                  type: Type.ARRAY,
                  items: { type: Type.STRING },
                },
                correctIndex: {
                  type: Type.INTEGER,
                  description: 'Zero-based index of the correct option (0-3).',
```

```
              },
              explanation: { type: Type.STRING },
            },
            required: ['question', 'options', 'correctIndex', 'explanation'],
          },
        },
        required: [
          'topic',
          'level',
          'simpleDefinition',
          'analogy',
          'keyPoints',
          'realWorldExample',
          'recap',
          'quickCheck',
        ],
      },
    },
  });

  const text = response.text || '{}';
  const parsed = JSON.parse(text);
  return res.json(parsed);
 } catch (error: any) {
  return handleGeminiError(error, res);
 }
});

// ---------------------------------------------------------------------------
// 3. Notes Summarizer (Text or uploaded .txt / .pdf file)
// ---------------------------------------------------------------------------
app.post('/api/summarize', async (req: Request, res: Response) => {
 const { text, fileData, format } = req.body;

 if (!text && !fileData) {
  return res.status(400).json({ error: 'Please provide either text or upload a file.' });
 }

 const selectedFormat = format || 'short_summary'; // 'short_summary' | 'bullet_points' | 'key_terms'

 let formatInstruction = '';
 if (selectedFormat === 'short_summary') {
  formatInstruction = 'Provide a structured, crystal-clear executive summary with an overview paragraph, major takeaways, and practica
 } else if (selectedFormat === 'bullet_points') {
  formatInstruction = 'Provide organized bullet points categorized into logical sections with descriptive sub-headings. Use bold text
 } else if (selectedFormat === 'key_terms') {
  formatInstruction = 'Extract all critical terms, formulas, key people/dates, or specialized vocabulary. Format each item with the Te
 }

 const systemInstruction = `You are EduGenie's Study Notes Summarizer. Your goal is to synthesize student notes, textbooks, and documen
Output strictly formatted Markdown. Format style: ${selectedFormat}.
${formatInstruction}`;

 try {
  const parts: any[] = [];

  // Support uploaded PDF or TXT files via Gemini inlineData
  if (fileData && fileData.base64 && fileData.mimeType) {
   parts.push({
    inlineData: {
     mimeType: fileData.mimeType,
     data: fileData.base64,
    },
   });
   parts.push({
    text: `Please summarize the contents of the attached document according to the requested format: ${selectedFormat}.${text ? ` Ad
   });
  } else {
   parts.push({
    text: `Here are the notes to summarize:\n\n${text}\n\nFormat requested: ${selectedFormat}.`,
   });
  }

  const response = await ai.models.generateContent({
   model: MODEL_NAME,
   contents: { parts },
   config: {
    systemInstruction,
    temperature: 0.3,
   },
  });

  const summaryText = response.text || '';
  return res.json({
   summary: summaryText,
   format: selectedFormat,
  });
 } catch (error: any) {
  return handleGeminiError(error, res);
```

```
  }
});

// ----------------------------------------------------------------------------
// 4. Quiz Generator (Structured Multiple Choice Questions)
// ----------------------------------------------------------------------------
app.post('/api/quiz', async (req: Request, res: Response) => {
  const { topic, notes, count = 5, difficulty = 'Medium' } = req.body;

  if (!topic && !notes) {
    return res.status(400).json({ error: 'Please provide a topic or notes for the quiz.' });
  }

  const questionCount = count === 10 ? 10 : 5;
  const prompt = topic
    ? `Generate ${questionCount} multiple-choice questions on "${topic}" at a ${difficulty} difficulty level.`
    : `Generate ${questionCount} multiple-choice questions based on the following notes at a ${difficulty} difficulty level:\n\n${notes}

  try {
    const response = await ai.models.generateContent({
      model: MODEL_NAME,
      contents: prompt,
      config: {
        systemInstruction: `You are EduGenie's Quiz Generator. Create exactly ${questionCount} high-quality, pedagogically sound multipl
Each question must have:
- Exactly 4 distinct, plausible options.
- Exactly one correct answer specified by index (0, 1, 2, or 3).
- A thorough, encouraging explanation explaining why the correct option is right and clarifying common student misconceptions.
Return strictly valid JSON matching the schema.`,
        responseMimeType: 'application/json',
        responseSchema: {
          type: Type.OBJECT,
          properties: {
            topic: { type: Type.STRING },
            difficulty: { type: Type.STRING },
            questions: {
              type: Type.ARRAY,
              items: {
                type: Type.OBJECT,
                properties: {
                  id: { type: Type.INTEGER },
                  question: { type: Type.STRING },
                  options: {
                    type: Type.ARRAY,
                    items: { type: Type.STRING },
                    description: 'Exactly 4 distinct answer choices.',
                  },
                  correctIndex: {
                    type: Type.INTEGER,
                    description: 'The 0-based index of the correct option (0, 1, 2, or 3).',
                  },
                  explanation: {
                    type: Type.STRING,
                    description: 'Explanation of why this choice is correct.',
                  },
                },
                required: ['id', 'question', 'options', 'correctIndex', 'explanation'],
              },
            },
          },
          required: ['topic', 'difficulty', 'questions'],
        },
      },
    });

    const text = response.text || '{}';
    const parsed = JSON.parse(text);
    return res.json(parsed);
  } catch (error: any) {
    return handleGeminiError(error, res);
  }
});

// ----------------------------------------------------------------------------
// 5. Flashcards Generator
// ----------------------------------------------------------------------------
app.post('/api/flashcards', async (req: Request, res: Response) => {
  const { topic, notes, count = 8 } = req.body;

  if (!topic && !notes) {
    return res.status(400).json({ error: 'Please provide a topic or notes to generate flashcards.' });
  }

  const flashcardCount = Math.min(Math.max(Number(count) || 8, 4), 16);
  const prompt = topic
    ? `Generate ${flashcardCount} smart flashcards for "${topic}".`
    : `Generate ${flashcardCount} smart flashcards based on these notes:\n\n${notes}`;

  try {
```

```
    const response = await ai.models.generateContent({
      model: MODEL_NAME,
      contents: prompt,
      config: {
        systemInstruction: `You are EduGenie's Flashcard Generator. Create effective spaced-repetition study flashcards.
Front should contain a focused question, prompt, or term.
Back should contain a clear, concise, accurate answer or definition with key takeaways.
Include a helpful "hint" and a "category" or topic tag for each card.
Output strictly JSON matching the schema.`,
        responseMimeType: 'application/json',
        responseSchema: {
          type: Type.OBJECT,
          properties: {
            deckTitle: { type: Type.STRING },
            cards: {
              type: Type.ARRAY,
              items: {
                type: Type.OBJECT,
                properties: {
                  id: { type: Type.STRING },
                  front: { type: Type.STRING, description: 'Question, concept, or term.' },
                  back: { type: Type.STRING, description: 'Answer, explanation, or definition.' },
                  hint: { type: Type.STRING, description: 'A subtle memory clue or hint.' },
                  category: { type: Type.STRING, description: 'Subtopic or category tag.' },
                },
                required: ['id', 'front', 'back', 'hint', 'category'],
              },
            },
          },
          required: ['deckTitle', 'cards'],
        },
      },
    });

    const text = response.text || '{}';
    const parsed = JSON.parse(text);
    return res.json(parsed);
  } catch (error: any) {
    return handleGeminiError(error, res);
  }
});

// ---------------------------------------------------------------------------
// 6. Study Planner Generator
// ---------------------------------------------------------------------------
app.post('/api/planner', async (req: Request, res: Response) => {
  const { subjects, examDate, hoursPerDay, pace = 'Balanced' } = req.body;

  if (!subjects || (Array.isArray(subjects) && subjects.length === 0)) {
    return res.status(400).json({ error: 'Please specify subjects or topics.' });
  }

  const subjectsList = Array.isArray(subjects) ? subjects.join(', ') : String(subjects);
  const prompt = `Create a complete day-by-day study schedule for a student preparing for exams.
Subjects/Topics: ${subjectsList}
Target Exam Date or Deadline: ${examDate || 'In 14 days'}
Available Study Hours per Day: ${hoursPerDay || 3} hours
Preferred Study Pace: ${pace}`;

  try {
    const response = await ai.models.generateContent({
      model: MODEL_NAME,
      contents: prompt,
      config: {
        systemInstruction: `You are EduGenie's Academic Study Planner. Create a realistic, science-backed study schedule incorporating a
Include dedicated revision days, practice test days, rest intervals, and manageable daily study blocks.
Format as strictly valid JSON matching the schema.`,
        responseMimeType: 'application/json',
        responseSchema: {
          type: Type.OBJECT,
          properties: {
            planTitle: { type: Type.STRING },
            overview: { type: Type.STRING, description: 'Brief strategy summary and study advice.' },
            totalDays: { type: Type.INTEGER },
            dailyHoursTarget: { type: Type.NUMBER },
            days: {
              type: Type.ARRAY,
              items: {
                type: Type.OBJECT,
                properties: {
                  dayNumber: { type: Type.INTEGER },
                  dateOrLabel: { type: Type.STRING, description: 'e.g., "Day 1 (Mon)" or specific date' },
                  theme: { type: Type.STRING, description: 'Core focus or subject for the day' },
                  isRevisionDay: { type: Type.BOOLEAN },
                  blocks: {
                    type: Type.ARRAY,
                    items: {
                      type: Type.OBJECT,
                      properties: {
```

```
                        timeSlot: { type: Type.STRING, description: 'e.g. "Morning (1.5 hrs)" or "Block 1"' },
                        subject: { type: Type.STRING },
                        task: { type: Type.STRING, description: 'Specific action or chapters to cover' },
                        technique: { type: Type.STRING, description: 'e.g., "Active Recall", "Flashcards", "Practice Problems"' },
                      },
                      required: ['timeSlot', 'subject', 'task', 'technique'],
                    },
                  },
                  dailyTip: { type: Type.STRING, description: 'A quick motivational or cognitive retention tip' },
                },
                required: ['dayNumber', 'dateOrLabel', 'theme', 'isRevisionDay', 'blocks', 'dailyTip'],
              },
            },
          },
          required: ['planTitle', 'overview', 'totalDays', 'dailyHoursTarget', 'days'],
        },
      },
    });

    const text = response.text || '{}';
    const parsed = JSON.parse(text);
    return res.json(parsed);
  } catch (error: any) {
    return handleGeminiError(error, res);
  }
});

// ---------------------------------------------------------------------------
// Vite Dev Server or Production Static Serving
// ---------------------------------------------------------------------------
async function startServer() {
  const isProd = process.env.NODE_ENV === 'production';

  if (!isProd) {
    const { createServer: createViteServer } = await import('vite');
    const vite = await createViteServer({
      server: { middlewareMode: true },
      appType: 'spa',
    });
    app.use(vite.middlewares);
  } else {
    const distPath = path.resolve(__dirname, 'dist');
    app.use(express.static(distPath));
    app.get('*', (_req: Request, res: Response) => {
      res.sendFile(path.join(distPath, 'index.html'));
    });
  }

  app.listen(PORT, '0.0.0.0', () => {
    console.log(`EduGenie server running on http://0.0.0.0:${PORT}`);
  });
}

startServer().catch((err) => {
  console.error('Failed to start server:', err);
  process.exit(1);
});
```

src/App.tsx

```
import React, { useState, useEffect } from 'react';
import { TabType, ChatMessage } from './types';
import {
  getStoredTheme,
  saveStoredTheme,
  getStoredChat,
  saveStoredChat,
  getStoredQuizHistory,
  getStoredFlashcards,
  clearAllUserData,
} from './utils/storage';
import { Header } from './components/common/Header';
import { Sidebar } from './components/common/Sidebar';
import { Footer } from './components/common/Footer';
import { TutorChat } from './components/tutor/TutorChat';
import { ConceptExplainer } from './components/explainer/ConceptExplainer';
import { NotesSummarizer } from './components/summarizer/NotesSummarizer';
import { QuizGenerator } from './components/quiz/QuizGenerator';
import { FlashcardDeck } from './components/flashcards/FlashcardDeck';
import { StudyPlanner } from './components/planner/StudyPlanner';

export default function App() {
  const [activeTab, setActiveTab] = useState<TabType>('tutor');
  const [theme, setTheme] = useState<'light' | 'dark'>(() => getStoredTheme());
  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false);

  // Persistent States
  const [chatMessages, setChatMessages] = useState<ChatMessage[]>(() => getStoredChat());
  const [quizHistoryCount, setQuizHistoryCount] = useState<number>(() => getStoredQuizHistory().length);
  const [flashcardCount, setFlashcardCount] = useState<number>(() => {
    const deck = getStoredFlashcards();
    return deck ? deck.cards.length : 0;
  });

  // Sync theme changes with DOM class and storage
  useEffect(() => {
    const root = document.documentElement;
    if (theme === 'dark') {
      root.classList.add('dark');
    } else {
      root.classList.remove('dark');
    }
    saveStoredTheme(theme);
  }, [theme]);

  // Sync chat messages to localStorage
  useEffect(() => {
    saveStoredChat(chatMessages);
  }, [chatMessages]);

  const toggleTheme = () => {
    setTheme((prev) => (prev === 'light' ? 'dark' : 'light'));
  };

  const handleClearSession = () => {
    clearAllUserData();
    setChatMessages([]);
    setQuizHistoryCount(0);
    setFlashcardCount(0);
    window.location.reload();
  };

  return (
    <div className="min-h-screen flex flex-col bg-slate-50 dark:bg-slate-950 text-slate-800 dark:text-slate-100 transition-colors">
      {/* Top Header */}
      <Header
        activeTab={activeTab}
        theme={theme}
        toggleTheme={toggleTheme}
        toggleMobileMenu={() => setIsMobileMenuOpen((prev) => !prev)}
      />

      <div className="flex-1 flex max-w-7xl mx-auto w-full">
        {/* Navigation Sidebar */}
        <Sidebar
          activeTab={activeTab}
          setActiveTab={setActiveTab}
          isOpen={isMobileMenuOpen}
          onClose={() => setIsMobileMenuOpen(false)}
          onClearSession={handleClearSession}
          chatCount={chatMessages.length}
          flashcardCount={flashcardCount}
          quizCount={quizHistoryCount}
        />

        {/* Main Content Area */}
        <main className="flex-1 p-4 sm:p-6 lg:p-8 overflow-y-auto">
```

```
          {activeTab === 'tutor' && (
            <TutorChat
              messages={chatMessages}
              setMessages={setChatMessages}
            />
          )}

          {activeTab === 'explainer' && (
            <ConceptExplainer />
          )}

          {activeTab === 'summarizer' && (
            <NotesSummarizer />
          )}

          {activeTab === 'quiz' && (
            <QuizGenerator
              onQuizCompleted={() => setQuizHistoryCount(getStoredQuizHistory().length)}
            />
          )}

          {activeTab === 'flashcards' && (
            <FlashcardDeck
              onDeckUpdated={() => {
                const deck = getStoredFlashcards();
                setFlashcardCount(deck ? deck.cards.length : 0);
              }}
            />
          )}

          {activeTab === 'planner' && (
            <StudyPlanner />
          )}
        </main>
      </div>

      {/* Footer with mandatory disclaimer */}
      <Footer />
    </div>
  );
}
```

src/components/common/ErrorMessage.tsx

```
import React, { useEffect, useState } from 'react';
import { AlertCircle, RotateCcw, Clock } from 'lucide-react';

interface ErrorMessageProps {
  message: string;
  isRateLimit?: boolean;
  onRetry?: () => void;
  cooldownSeconds?: number;
}

export const ErrorMessage: React.FC<ErrorMessageProps> = ({
  message,
  isRateLimit = false,
  onRetry,
  cooldownSeconds = 3,
}) => {
  const [secondsLeft, setSecondsLeft] = useState(isRateLimit ? Math.max(cooldownSeconds, 5) : cooldownSeconds);

  useEffect(() => {
    if (secondsLeft <= 0) return;
    const timer = setInterval(() => {
      setSecondsLeft((prev) => Math.max(prev - 1, 0));
    }, 1000);
    return () => clearInterval(timer);
  }, [secondsLeft]);

  return (
    <div
      role="alert"
      className="p-4 my-3 rounded-2xl border border-amber-200 dark:border-amber-900/60 bg-amber-50 dark:bg-amber-950/40 text-amber-900 d
    >
      <div className="flex items-start gap-3">
        <div className="p-2 rounded-xl bg-amber-100 dark:bg-amber-900/50 text-amber-600 dark:text-amber-400 shrink-0">
          <AlertCircle className="w-5 h-5" />
        </div>
        <div className="flex-1 min-w-0">
          <h4 className="font-semibold text-sm text-amber-950 dark:text-amber-100">
            {isRateLimit ? 'Rate Limit Reached' : 'Unable to complete request'}
          </h4>
          <p className="text-sm mt-0.5 text-amber-800 dark:text-amber-300/90 leading-relaxed">
            {message}
          </p>

          {onRetry && (
            <div className="mt-3 flex items-center gap-3">
              <button
                type="button"
                onClick={onRetry}
                disabled={secondsLeft > 0}
                className="inline-flex items-center gap-1.5 px-3 py-1.5 text-xs font-semibold rounded-lg bg-amber-600 hover:bg-amber-700
              >
                <RotateCcw className={`w-3.5 h-3.5 ${secondsLeft > 0 ? 'animate-spin' : ''}`} />
                <span>Retry Now</span>
              </button>

              {secondsLeft > 0 && (
                <span className="inline-flex items-center gap-1 text-xs text-amber-750 dark:text-amber-400 font-medium">
                  <Clock className="w-3.5 h-3.5" />
                  <span>Wait {secondsLeft}s cooldown</span>
                </span>
              )}
            </div>
          )}
        </div>
      </div>
    </div>
  );
};
```

src/components/common/Footer.tsx

```
import React from 'react';
import { AlertCircle, ShieldCheck, Heart } from 'lucide-react';

export const Footer: React.FC = () => {
  return (
    <footer className="mt-auto py-5 px-4 border-t border-slate-200/80 dark:border-slate-800/80 bg-white/50 dark:bg-slate-900/50 text-xs
      <div className="max-w-7xl mx-auto flex flex-col sm:flex-row items-center justify-between gap-3 text-center sm:text-left">
        {/* Mandatory Disclaimer */}
        <div className="flex items-center gap-1.5 text-amber-700 dark:text-amber-400/90 font-medium">
          <AlertCircle className="w-4 h-4 shrink-0 text-amber-500" />
          <span>AI can make mistakes. Verify important information.</span>
        </div>

        {/* Free Tier & Privacy Badge */}
        <div className="flex items-center gap-4 text-slate-500 dark:text-slate-400">
          <span className="flex items-center gap-1">
            <ShieldCheck className="w-3.5 h-3.5 text-emerald-500" />
            <span>Private & Browser-Stored</span>
          </span>
          <span>•</span>
          <span className="flex items-center gap-1">
            Built for learners with <Heart className="w-3 h-3 text-rose-500 fill-rose-500" />
          </span>
        </div>
      </div>
    </footer>
  );
};
```

src/components/common/Header.tsx

```
import React from 'react';
import { Sparkles, Sun, Moon, Menu, GraduationCap } from 'lucide-react';
import { TabType } from '../../types';

interface HeaderProps {
  activeTab: TabType;
  theme: 'light' | 'dark';
  toggleTheme: () => void;
  toggleMobileMenu: () => void;
}

const TAB_TITLES: Record<TabType, { title: string; subtitle: string }> = {
  tutor: { title: 'AI Tutor Chat', subtitle: 'Step-by-step doubt solver & study mentor' },
  explainer: { title: 'Concept Explainer', subtitle: 'Deep understanding with analogies & examples' },
  summarizer: { title: 'Notes Summarizer', subtitle: 'Transform notes and PDFs into key takeaways' },
  quiz: { title: 'Quiz Generator', subtitle: 'Test your mastery with instant explanations' },
  flashcards: { title: 'Interactive Flashcards', subtitle: 'Spaced repetition cards for active recall' },
  planner: { title: 'Study Planner', subtitle: 'Personalized exam schedule with revision days' },
};

export const Header: React.FC<HeaderProps> = ({
  activeTab,
  theme,
  toggleTheme,
  toggleMobileMenu,
}) => {
  const currentTab = TAB_TITLES[activeTab];

  return (
    <header className="sticky top-0 z-30 bg-white/80 dark:bg-slate-900/80 backdrop-blur-md border-b border-slate-200/80 dark:border-slat
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 h-16 flex items-center justify-between gap-4">
        {/* Left: Mobile Menu Toggle + App Branding / Page Header */}
        <div className="flex items-center gap-3">
          <button
            type="button"
            onClick={toggleMobileMenu}
            className="md:hidden p-2 rounded-xl text-slate-600 dark:text-slate-300 hover:bg-slate-100 dark:hover:bg-slate-800 focus:outl
            aria-label="Open menu"
          >
            <Menu className="w-5 h-5" />
          </button>

          <div className="flex items-center gap-2.5">
            <div className="w-9 h-9 rounded-xl bg-gradient-to-tr from-indigo-600 via-indigo-500 to-sky-400 flex items-center justify-cer
              <GraduationCap className="w-5 h-5" />
            </div>
            <div>
              <div className="flex items-center gap-1.5">
                <span className="font-extrabold text-base tracking-tight bg-gradient-to-r from-indigo-600 to-sky-500 bg-clip-text text-t
                  EduGenie
                </span>
                <span className="hidden sm:inline-flex items-center gap-1 px-2 py-0.5 rounded-full text-[10px] font-semibold tracking-wi
                  <Sparkles className="w-2.5 h-2.5" />
                  Gemini Flash
                </span>
              </div>
              <p className="hidden md:block text-xs text-slate-500 dark:text-slate-400 font-medium">
                {currentTab.title} — {currentTab.subtitle}
              </p>
            </div>
          </div>
        </div>

        {/* Right: Theme Toggle & Quick Status */}
        <div className="flex items-center gap-2">
          <div className="hidden lg:flex items-center gap-1.5 px-3 py-1 rounded-full bg-slate-100 dark:bg-slate-800 text-xs font-medium
            <span className="w-2 h-2 rounded-full bg-emerald-500 animate-pulse"></span>
            <span>Free Tier • Ready</span>
          </div>

          <button
            type="button"
            onClick={toggleTheme}
            className="p-2.5 rounded-xl border border-slate-200/80 dark:border-slate-700/80 bg-slate-50 dark:bg-slate-800/80 text-slate-
            aria-label={`Switch to ${theme === 'dark' ? 'light' : 'dark'} mode`}
            title={`Switch to ${theme === 'dark' ? 'light' : 'dark'} mode`}
          >
            {theme === 'dark' ? (
              <Sun className="w-4 h-4 text-amber-400 hover:rotate-90 transition-transform duration-300" />
            ) : (
              <Moon className="w-4 h-4 text-slate-600 hover:-rotate-12 transition-transform duration-300" />
            )}
          </button>
        </div>
      </div>
    </header>
  );
```

```
};
```

src/components/common/MarkdownRenderer.tsx

```
import React, { useState } from 'react';
import ReactMarkdown from 'react-markdown';
import remarkGfm from 'remark-gfm';
import { Copy, Check } from 'lucide-react';

interface MarkdownRendererProps {
  content: string;
  className?: string;
}

export const MarkdownRenderer: React.FC<MarkdownRendererProps> = ({ content, className = '' }) => {
  return (
    <div className={`prose dark:prose-invert max-w-none text-slate-800 dark:text-slate-200 leading-relaxed ${className}`}>
      <ReactMarkdown
        remarkPlugins={[remarkGfm]}
        components={{
          code({ className, children, ...props }) {
            const match = /language-(\w+)/.exec(className || '');
            const isInline = !match && !String(children).includes('\n');

            if (isInline) {
              return (
                <code
                  className="bg-indigo-50 dark:bg-indigo-950/70 text-indigo-700 dark:text-indigo-300 font-mono text-sm px-1.5 py-0.5 rou
                  {...props}
                >
                  {children}
                </code>
              );
            }

            return <CodeBlock language={match ? match[1] : ''}>{String(children).replace(/\n$/, '')}</CodeBlock>;
          },
          h1({ children }) {
            return <h1 className="text-xl sm:text-2xl font-bold text-slate-900 dark:text-white mt-4 mb-2 tracking-tight">{children}</h1>
          },
          h2({ children }) {
            return <h2 className="text-lg sm:text-xl font-bold text-slate-900 dark:text-white mt-3 mb-2 tracking-tight">{children}</h2>;
          },
          h3({ children }) {
            return <h3 className="text-base sm:text-lg font-semibold text-slate-800 dark:text-slate-100 mt-2.5 mb-1.5">{children}</h3>;
          },
          ul({ children }) {
            return <ul className="list-disc pl-5 my-2 space-y-1 text-slate-700 dark:text-slate-300">{children}</ul>;
          },
          ol({ children }) {
            return <ol className="list-decimal pl-5 my-2 space-y-1 text-slate-700 dark:text-slate-300">{children}</ol>;
          },
          li({ children }) {
            return <li className="leading-relaxed">{children}</li>;
          },
          p({ children }) {
            return <p className="my-2 leading-relaxed">{children}</p>;
          },
          blockquote({ children }) {
            return (
              <blockquote className="border-l-4 border-indigo-500 bg-indigo-50/50 dark:bg-indigo-950/30 pl-4 py-1.5 my-3 italic text-sla
                {children}
              </blockquote>
            );
          },
          table({ children }) {
            return (
              <div className="overflow-x-auto my-3 rounded-lg border border-slate-200 dark:border-slate-800">
                <table className="w-full text-left text-sm divide-y divide-slate-200 dark:divide-slate-800">
                  {children}
                </table>
              </div>
            );
          },
          th({ children }) {
            return <th className="bg-slate-100 dark:bg-slate-800/80 px-4 py-2 font-semibold text-slate-900 dark:text-white">{children}</
          },
          td({ children }) {
            return <td className="px-4 py-2 border-t border-slate-100 dark:border-slate-800/50 text-slate-700 dark:text-slate-300">{chil
          },
        }}
      >
        {content}
      </ReactMarkdown>
    </div>
  );
};

const CodeBlock: React.FC<{ language: string; children: string }> = ({ language, children }) => {
  const [copied, setCopied] = useState(false);
```

```
  const handleCopy = () => {
    navigator.clipboard.writeText(children);
    setCopied(true);
    setTimeout(() => setCopied(false), 2000);
  };

  return (
    <div className="relative group my-3 rounded-xl overflow-hidden border border-slate-200 dark:border-slate-800 bg-slate-900 text-slate
      <div className="flex items-center justify-between px-4 py-1.5 bg-slate-800/90 text-xs font-mono text-slate-400 border-b border-sla
        <span>{language || 'code'}</span>
        <button
          onClick={handleCopy}
          className="flex items-center gap-1 text-slate-400 hover:text-white transition-colors py-0.5 px-2 rounded hover:bg-slate-700/60
          title="Copy code"
          type="button"
        >
          {copied ? (
            <>
              <Check className="w-3.5 h-3.5 text-emerald-400" />
              <span className="text-emerald-400">Copied!</span>
            </>
          ) : (
            <>
              <Copy className="w-3.5 h-3.5" />
              <span>Copy</span>
            </>
          )}
        </button>
      </div>
      <pre className="p-4 overflow-x-auto text-sm font-mono leading-relaxed text-slate-100">
        <code>{children}</code>
      </pre>
    </div>
  );
};
```

src/components/common/Sidebar.tsx

```
import React, { useState } from 'react';
import {
  MessageSquare,
  Lightbulb,
  FileText,
  HelpCircle,
  Layers,
  CalendarDays,
  Trash2,
  X,
  GraduationCap,
  Sparkles,
  BookOpen,
} from 'lucide-react';
import { TabType } from '../../types';

interface SidebarProps {
  activeTab: TabType;
  setActiveTab: (tab: TabType) => void;
  isOpen: boolean;
  onClose: () => void;
  onClearSession: () => void;
  chatCount: number;
  flashcardCount: number;
  quizCount: number;
}

interface NavItem {
  id: TabType;
  label: string;
  description: string;
  icon: React.ElementType;
  badge?: string | number;
}

export const Sidebar: React.FC<SidebarProps> = ({
  activeTab,
  setActiveTab,
  isOpen,
  onClose,
  onClearSession,
  chatCount,
  flashcardCount,
  quizCount,
}) => {
  const [showClearConfirm, setShowClearConfirm] = useState(false);

  const navItems: NavItem[] = [
    {
      id: 'tutor',
      label: 'AI Tutor Chat',
      description: 'Doubt solver & explanation',
      icon: MessageSquare,
      badge: chatCount > 0 ? chatCount : undefined,
    },
    {
      id: 'explainer',
      label: 'Concept Explainer',
      description: 'Analogies & breakdowns',
      icon: Lightbulb,
    },
    {
      id: 'summarizer',
      label: 'Notes Summarizer',
      description: 'Text & PDF distillations',
      icon: FileText,
    },
    {
      id: 'quiz',
      label: 'Quiz Generator',
      description: 'Instant practice & grading',
      icon: HelpCircle,
      badge: quizCount > 0 ? `${quizCount} taken` : undefined,
    },
    {
      id: 'flashcards',
      label: 'Flashcards',
      description: 'Flip-card active recall',
      icon: Layers,
      badge: flashcardCount > 0 ? `${flashcardCount}` : undefined,
    },
    {
      id: 'planner',
      label: 'Study Planner',
      description: 'Exam timetable & revision',
      icon: CalendarDays,
    },
```

```
];

const handleSelectTab = (tab: TabType) => {
  setActiveTab(tab);
  onClose();
};

const handleConfirmClear = () => {
  onClearSession();
  setShowClearConfirm(false);
};

return (
  <>
    {/* Mobile Backdrop */}
    {isOpen && (
      <div
        className="fixed inset-0 z-40 bg-slate-900/60 backdrop-blur-xs md:hidden"
        onClick={onClose}
      />
    )}

    {/* Sidebar Container */}
    <aside
      className={`fixed md:sticky top-0 md:top-16 z-40 md:z-20 h-full md:h-[calc(100vh-4rem)] w-72 shrink-0 bg-white dark:bg-slate-900
        isOpen ? 'translate-x-0' : '-translate-x-full md:translate-x-0'
      }`}
    >
      {/* Top Header in Mobile */}
      <div className="p-4 flex items-center justify-between border-b border-slate-200/80 dark:border-slate-800/80 md:hidden">
        <div className="flex items-center gap-2">
          <div className="w-8 h-8 rounded-lg bg-indigo-600 flex items-center justify-center text-white">
            <GraduationCap className="w-5 h-5" />
          </div>
          <span className="font-bold text-slate-900 dark:text-white">EduGenie</span>
        </div>
        <button
          type="button"
          onClick={onClose}
          className="p-1.5 rounded-lg text-slate-500 hover:bg-slate-100 dark:hover:bg-slate-800"
        >
          <X className="w-5 h-5" />
        </button>
      </div>

      {/* Navigation Links */}
      <div className="p-3.5 space-y-1.5 overflow-y-auto flex-1">
        <div className="px-3 py-2 text-[11px] font-bold uppercase tracking-wider text-slate-400 dark:text-slate-500 flex items-center
          <BookOpen className="w-3.5 h-3.5" />
          <span>Learning Suite</span>
        </div>

        {navItems.map((item) => {
          const Icon = item.icon;
          const isActive = activeTab === item.id;
          return (
            <button
              key={item.id}
              type="button"
              onClick={() => handleSelectTab(item.id)}
              className={`w-full text-left px-3.5 py-3 rounded-xl transition-all flex items-center justify-between group cursor-pointe
                isActive
                  ? 'bg-gradient-to-r from-indigo-50 to-sky-50 dark:from-indigo-950/60 dark:to-slate-850 text-indigo-700 dark:text-ind
                  : 'text-slate-600 dark:text-slate-300 hover:bg-slate-100/80 dark:hover:bg-slate-800/60 hover:text-slate-900 dark:hov
              }`}
            >
              <div className="flex items-center gap-3 min-w-0">
                <div
                  className={`p-2 rounded-lg transition-colors ${
                    isActive
                      ? 'bg-indigo-600 text-white shadow-xs'
                      : 'bg-slate-100 dark:bg-slate-800 text-slate-500 dark:text-slate-400 group-hover:text-indigo-600 dark:group-hove
                  }`}
                >
                  <Icon className="w-4 h-4" />
                </div>
                <div className="truncate">
                  <div className="text-sm font-medium leading-tight">{item.label}</div>
                  <div className="text-[11px] text-slate-400 dark:text-slate-500 truncate mt-0.5">
                    {item.description}
                  </div>
                </div>
              </div>

              {item.badge !== undefined && (
                <span
                  className={`ml-2 px-2 py-0.5 rounded-full text-[10px] font-semibold shrink-0 ${
                    isActive
                      ? 'bg-indigo-600 text-white'
```

```
                    : 'bg-slate-200/70 dark:bg-slate-800 text-slate-600 dark:text-slate-400'
                }`}
              >
                {item.badge}
              </span>
            )}
          </button>
        );
      })}
    </div>

    {/* Bottom Actions: Clear Session & Study Tip */}
    <div className="p-3.5 border-t border-slate-200/80 dark:border-slate-800/80 bg-slate-50/50 dark:bg-slate-900/50 space-y-2.5">
      <div className="p-3 rounded-xl bg-gradient-to-br from-indigo-500/10 via-sky-500/5 to-transparent border border-indigo-200/50 d
        <div className="flex items-center gap-1.5 text-xs font-semibold text-indigo-700 dark:text-indigo-300 mb-1">
          <Sparkles className="w-3.5 h-3.5" />
          <span>Student Tip</span>
        </div>
        <p className="text-[11px] text-slate-600 dark:text-slate-400 leading-normal">
          Test your knowledge right after reading with Flashcards or Quiz to double retention!
        </p>
      </div>

      {/* Clear Session Button */}
      <button
        type="button"
        onClick={() => setShowClearConfirm(true)}
        className="w-full flex items-center justify-center gap-2 px-3 py-2 rounded-xl text-xs font-medium text-rose-600 dark:text-ro
        title="Reset and clear all saved local session data"
      >
        <Trash2 className="w-3.5 h-3.5" />
        <span>Clear Session Data</span>
      </button>
    </div>
  </aside>

  {/* Confirmation Modal */}
  {showClearConfirm && (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-slate-900/60 backdrop-blur-xs animate-fadeIn">
      <div className="bg-white dark:bg-slate-900 rounded-2xl p-6 max-w-sm w-full shadow-2xl border border-slate-200 dark:border-slat
        <div className="w-12 h-12 rounded-2xl bg-rose-100 dark:bg-rose-950/60 text-rose-600 dark:text-rose-400 flex items-center jus
          <Trash2 className="w-6 h-6" />
        </div>
        <h3 className="text-lg font-bold text-center text-slate-900 dark:text-white">
          Clear All Session Data?
        </h3>
        <p className="text-sm text-center text-slate-500 dark:text-slate-400 mt-2 leading-relaxed">
          This will reset your chat history, saved quiz records, generated flashcards, and study plans in this browser.
        </p>
        <div className="mt-6 flex gap-3">
          <button
            type="button"
            onClick={() => setShowClearConfirm(false)}
            className="flex-1 px-4 py-2.5 rounded-xl border border-slate-200 dark:border-slate-700 text-sm font-semibold text-slate-
          >
            Cancel
          </button>
          <button
            type="button"
            onClick={handleConfirmClear}
            className="flex-1 px-4 py-2.5 rounded-xl bg-rose-600 hover:bg-rose-700 text-sm font-semibold text-white shadow-sm transi
          >
            Clear All
          </button>
        </div>
      </div>
    </div>
  )}
</>
);
};
```

src/components/explainer/ConceptExplainer.tsx

```
import React, { useState } from 'react';
import {
  Lightbulb,
  Sparkles,
  CheckCircle2,
  Globe,
  RotateCcw,
  BookOpen,
  ArrowRight,
  Layers,
  HelpCircle,
  Copy,
  Check,
} from 'lucide-react';
import { ConceptExplanation } from '../../types';
import { ErrorMessage } from '../common/ErrorMessage';
import { saveStoredConcept, getStoredConcept } from '../../utils/storage';

const SAMPLE_TOPICS = [
  'Black Holes & Event Horizon',
  'Mitochondria & ATP Cellular Energy',
  'Supply and Demand in Economics',
  'Object-Oriented Programming (OOP)',
  'Plate Tectonics & Continental Drift',
  'The Doppler Effect in Sound & Light',
];

export const ConceptExplainer: React.FC = () => {
  const [topic, setTopic] = useState('');
  const [level, setLevel] = useState<'Beginner' | 'Intermediate' | 'Advanced'>('Intermediate');
  const [subject, setSubject] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [explanation, setExplanation] = useState<ConceptExplanation | null>(() => getStoredConcept());
  const [errorState, setErrorState] = useState<{ message: string; isRateLimit: boolean } | null>(null);

  // Quick check state
  const [selectedOption, setSelectedOption] = useState<number | null>(null);
  const [showAnswer, setShowAnswer] = useState(false);
  const [copied, setCopied] = useState(false);

  const handleExplain = async (customTopic?: string) => {
    const topicToSearch = (customTopic || topic).trim();
    if (!topicToSearch || isLoading) return;

    setIsLoading(true);
    setErrorState(null);
    setSelectedOption(null);
    setShowAnswer(false);

    try {
      const res = await fetch('/api/explain', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          topic: topicToSearch,
          level,
          subject: subject.trim(),
        }),
      });

      const data = await res.json();
      if (!res.ok) {
        throw new Error(data.error || 'Failed to explain concept.');
      }

      setExplanation(data);
      saveStoredConcept(data);
    } catch (err: any) {
      console.error('Explainer error:', err);
      const is429 = err.message?.includes('429') || err.message?.includes('Too many requests');
      setErrorState({
        message: err.message || 'Could not explain concept. Please check your network and try again.',
        isRateLimit: is429,
      });
    } finally {
      setIsLoading(false);
    }
  };

  const handleCopyExplanation = () => {
    if (!explanation) return;
    const text = `# ${explanation.topic} (${explanation.level} Level)\n\n## Simple Definition\n${explanation.simpleDefinition}\n\n## Eve
    navigator.clipboard.writeText(text);
    setCopied(true);
    setTimeout(() => setCopied(false), 2000);
  };
```

```
return (
  <div className="max-w-4xl mx-auto space-y-6">
    {/* Input Card */}
    <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 sm:p-8 border border-slate-200/80 dark:border-slate-800/80 shadow-sm tr
      <div className="flex items-center gap-3 mb-4">
        <div className="p-3 rounded-2xl bg-amber-100 dark:bg-amber-950/60 text-amber-600 dark:text-amber-400">
          <Lightbulb className="w-6 h-6" />
        </div>
        <div>
          <h2 className="text-xl font-bold text-slate-900 dark:text-white">
            Concept Explainer
          </h2>
          <p className="text-sm text-slate-500 dark:text-slate-400">
            Break down any confusing topic into clear definitions, intuitive analogies, and real-life examples.
          </p>
        </div>
      </div>

      <form
        onSubmit={(e) => {
          e.preventDefault();
          handleExplain();
        }}
        className="space-y-4"
      >
        <div>
          <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
            Concept or Topic to Master
          </label>
          <div className="relative">
            <input
              type="text"
              value={topic}
              onChange={(e) => setTopic(e.target.value)}
              placeholder="e.g. CRISPR gene editing, Bayes Theorem, Keynesian economics, Quantum tunneling..."
              className="w-full px-4 py-3 rounded-xl bg-slate-50 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700 te
            />
          </div>
        </div>

        <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
          {/* Level Selector */}
          <div>
            <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
              Explanation Depth
            </label>
            <div className="grid grid-cols-3 gap-1.5 bg-slate-100 dark:bg-slate-800 p-1 rounded-xl">
              {(['Beginner', 'Intermediate', 'Advanced'] as const).map((lvl) => (
                <button
                  key={lvl}
                  type="button"
                  onClick={() => setLevel(lvl)}
                  className={`py-2 px-2 text-xs font-semibold rounded-lg transition-all cursor-pointer ${
                    level === lvl
                      ? 'bg-white dark:bg-slate-700 text-indigo-600 dark:text-indigo-300 shadow-xs'
                      : 'text-slate-600 dark:text-slate-400 hover:text-slate-900 dark:hover:text-white'
                  }`}
                >
                  {lvl}
                </button>
              ))}
            </div>
          </div>

          {/* Optional Subject */}
          <div>
            <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
              Subject or Context (Optional)
            </label>
            <input
              type="text"
              value={subject}
              onChange={(e) => setSubject(e.target.value)}
              placeholder="e.g. AP Biology, Grade 10 Physics, CS 101"
              className="w-full px-4 py-2.5 rounded-xl bg-slate-50 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700
            />
          </div>
        </div>

        {/* Sample Prompts */}
        <div className="pt-2">
          <span className="text-xs text-slate-400 font-medium mr-2">Try sample:</span>
          <div className="inline-flex flex-wrap gap-1.5 mt-1">
            {SAMPLE_TOPICS.map((item, idx) => (
              <button
                key={idx}
                type="button"
                onClick={() => {
                  setTopic(item);
```

```
              handleExplain(item);
            }}
            className="px-2.5 py-1 rounded-lg text-xs bg-slate-100 hover:bg-indigo-50 hover:text-indigo-600 dark:bg-slate-800 dark
          >
            {item}
          </button>
        ))}
      </div>
    </div>

    <div className="pt-2 flex justify-end">
      <button
        type="submit"
        disabled={!topic.trim() || isLoading}
        className="px-6 py-3 rounded-xl bg-indigo-600 hover:bg-indigo-700 disabled:opacity-50 text-white font-semibold text-sm sha
      >
        {isLoading ? (
          <>
            <Sparkles className="w-4 h-4 animate-spin" />
            <span>Synthesizing Concept...</span>
          </>
        ) : (
          <>
            <Sparkles className="w-4 h-4" />
            <span>Explain Concept</span>
          </>
        )}
      </button>
    </div>
  </form>
</div>

{/* Error state */}
{errorState && (
  <ErrorMessage
    message={errorState.message}
    isRateLimit={errorState.isRateLimit}
    onRetry={() => handleExplain()}
  />
)}

{/* Output Display */}
{explanation && !isLoading && (
  <div className="space-y-4 animate-fadeIn">
    {/* Header Card with Copy */}
    <div className="flex items-center justify-between px-2">
      <div className="flex items-center gap-2">
        <span className="text-xs font-bold uppercase tracking-wider px-2.5 py-0.5 rounded-full bg-indigo-100 dark:bg-indigo-950/80
          {explanation.level} Level
        </span>
        <h3 className="text-xl font-extrabold text-slate-900 dark:text-white">
          {explanation.topic}
        </h3>
      </div>

      <button
        type="button"
        onClick={handleCopyExplanation}
        className="inline-flex items-center gap-1.5 px-3 py-1.5 rounded-xl border border-slate-200 dark:border-slate-700 text-xs f
      >
        {copied ? (
          <>
            <Check className="w-3.5 h-3.5 text-emerald-500" />
            <span className="text-emerald-600 dark:text-emerald-400">Copied!</span>
          </>
        ) : (
          <>
            <Copy className="w-3.5 h-3.5" />
            <span>Copy Notes</span>
          </>
        )}
      </button>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
      {/* 1. Simple Definition */}
      <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 border border-slate-200/80 dark:border-slate-800/80 shadow-xs fle
        <div>
          <div className="flex items-center gap-2 text-indigo-600 dark:text-indigo-400 font-bold text-sm mb-2">
            <BookOpen className="w-4 h-4" />
            <span>Simple Definition</span>
          </div>
          <p className="text-slate-700 dark:text-slate-300 text-sm leading-relaxed">
            {explanation.simpleDefinition}
          </p>
        </div>
      </div>

      {/* 2. Vivid Everyday Analogy */}
```

```
  <div className="bg-gradient-to-br from-amber-500/10 via-amber-500/5 to-transparent dark:from-amber-950/40 dark:to-slate-900/
    <div>
      <div className="flex items-center gap-2 text-amber-600 dark:text-amber-400 font-bold text-sm mb-2">
        <Lightbulb className="w-4 h-4" />
        <span>Everyday Analogy</span>
      </div>
      <p className="text-slate-700 dark:text-slate-300 text-sm leading-relaxed italic">
        "{explanation.analogy}"
      </p>
    </div>
  </div>
</div>

{/* 3. Key Points */}
<div className="bg-white dark:bg-slate-900 rounded-3xl p-6 sm:p-7 border border-slate-200/80 dark:border-slate-800/80 shadow-x
  <div className="flex items-center gap-2 text-sky-600 dark:text-sky-400 font-bold text-sm mb-3">
    <Layers className="w-4 h-4" />
    <span>Core Key Points</span>
  </div>
  <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
    {explanation.keyPoints.map((point, index) => (
      <div
        key={index}
        className="flex items-start gap-2.5 p-3 rounded-2xl bg-slate-50 dark:bg-slate-800/60 border border-slate-200/60 dark:b
      >
        <CheckCircle2 className="w-4 h-4 text-emerald-500 shrink-0 mt-0.5" />
        <span>{point}</span>
      </div>
    ))}
  </div>
</div>

{/* 4. Real World Example & Golden Recap */}
<div className="grid grid-cols-1 md:grid-cols-2 gap-4">
  <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 border border-slate-200/80 dark:border-slate-800/80 shadow-xs">
    <div className="flex items-center gap-2 text-emerald-600 dark:text-emerald-400 font-bold text-sm mb-2">
      <Globe className="w-4 h-4" />
      <span>Real-World Application</span>
    </div>
    <p className="text-slate-700 dark:text-slate-300 text-sm leading-relaxed">
      {explanation.realWorldExample}
    </p>
  </div>

  <div className="bg-gradient-to-br from-indigo-500/10 via-sky-500/5 to-transparent dark:from-indigo-950/40 dark:to-slate-900/
    <div className="flex items-center gap-2 text-indigo-600 dark:text-indigo-400 font-bold text-sm mb-2">
      <Sparkles className="w-4 h-4" />
      <span>Golden Recap</span>
    </div>
    <p className="text-slate-800 dark:text-slate-200 font-semibold text-sm leading-relaxed">
      {explanation.recap}
    </p>
  </div>
</div>

{/* 5. Quick Comprehension Check */}
{explanation.quickCheck && (
  <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 sm:p-7 border border-slate-200/80 dark:border-slate-800/80 shadow
    <div className="flex items-center gap-2 text-purple-600 dark:text-purple-400 font-bold text-sm mb-3">
      <HelpCircle className="w-4 h-4" />
      <span>Quick Check: Test Your Understanding</span>
    </div>
    <h4 className="text-base font-semibold text-slate-900 dark:text-white mb-3">
      {explanation.quickCheck.question}
    </h4>

    <div className="space-y-2 mb-4">
      {explanation.quickCheck.options.map((opt, idx) => {
        const isChosen = selectedOption === idx;
        const isCorrect = idx === explanation.quickCheck.correctIndex;
        let btnStyle = 'border-slate-200 dark:border-slate-700 hover:border-indigo-300 bg-slate-50 dark:bg-slate-800/60';

        if (showAnswer) {
          if (isCorrect) {
            btnStyle = 'border-emerald-500 bg-emerald-50 dark:bg-emerald-950/40 text-emerald-900 dark:text-emerald-200 font-se
          } else if (isChosen) {
            btnStyle = 'border-rose-500 bg-rose-50 dark:bg-rose-950/40 text-rose-900 dark:text-rose-200';
          }
        } else if (isChosen) {
          btnStyle = 'border-indigo-500 bg-indigo-50 dark:bg-indigo-950/50 text-indigo-900 dark:text-indigo-200 font-semibold'
        }

        return (
          <button
            key={idx}
            type="button"
            onClick={() => !showAnswer && setSelectedOption(idx)}
            className={`w-full text-left p-3 rounded-2xl border text-sm transition-all flex items-center justify-between cursc
          >
```

```
                    <span>{opt}</span>
                    {showAnswer && isCorrect && <CheckCircle2 className="w-4 h-4 text-emerald-500 shrink-0 ml-2" />}
                  </button>
                );
              })}
            </div>

            {!showAnswer ? (
              <button
                type="button"
                disabled={selectedOption === null}
                onClick={() => setShowAnswer(true)}
                className="px-5 py-2.5 rounded-xl bg-purple-600 hover:bg-purple-700 disabled:opacity-50 text-white font-semibold text-
              >
                Check Answer
              </button>
            ) : (
              <div className="p-4 rounded-2xl bg-slate-100 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700 text-xs
                <div className="font-semibold text-slate-900 dark:text-white mb-1">
                  {selectedOption === explanation.quickCheck.correctIndex ? '■ Spot on!' : '■ Good try!'}
                </div>
                <p className="text-slate-600 dark:text-slate-300">
                  {explanation.quickCheck.explanation}
                </p>
              </div>
            )}
          </div>
        )}
      </div>
    )}
  </div>
);
};
```

src/components/flashcards/FlashcardDeck.tsx

```
import React, { useState, useEffect } from 'react';
import {
  Layers,
  Sparkles,
  RotateCw,
  ChevronLeft,
  ChevronRight,
  CheckCircle2,
  HelpCircle,
  Shuffle,
  Plus,
  Trash2,
  BookmarkCheck,
  Eye,
} from 'lucide-react';
import { Flashcard, FlashcardDeck as FlashcardDeckType } from '../../types';
import { ErrorMessage } from '../common/ErrorMessage';
import { getStoredFlashcards, saveStoredFlashcards } from '../../utils/storage';

interface FlashcardDeckProps {
  onDeckUpdated?: () => void;
}

export const FlashcardDeck: React.FC<FlashcardDeckProps> = ({ onDeckUpdated }) => {
  const [topic, setTopic] = useState('');
  const [notes, setNotes] = useState('');
  const [inputMode, setInputMode] = useState<'topic' | 'notes'>('topic');
  const [count, setCount] = useState(8);
  const [isLoading, setIsLoading] = useState(false);
  const [errorState, setErrorState] = useState<{ message: string; isRateLimit: boolean } | null>(null);

  // Deck State
  const [deck, setDeck] = useState<FlashcardDeckType | null>(() => getStoredFlashcards());
  const [currentIndex, setCurrentIndex] = useState(0);
  const [isFlipped, setIsFlipped] = useState(false);
  const [filterMode, setFilterMode] = useState<'all' | 'unmastered' | 'mastered'>('all');
  const [showHint, setShowHint] = useState(false);

  // Keyboard navigation
  useEffect(() => {
    const handleKeyDown = (e: KeyboardEvent) => {
      if (e.target instanceof HTMLInputElement || e.target instanceof HTMLTextAreaElement) return;

      if (e.code === 'Space') {
        e.preventDefault();
        setIsFlipped((prev) => !prev);
      } else if (e.key === 'ArrowRight') {
        e.preventDefault();
        handleNext();
      } else if (e.key === 'ArrowLeft') {
        e.preventDefault();
        handlePrev();
      }
    }

    window.addEventListener('keydown', handleKeyDown);
    return () => window.removeEventListener('keydown', handleKeyDown);
  });

  const handleGenerateDeck = async () => {
    const payloadTopic = inputMode === 'topic' ? topic.trim() : '';
    const payloadNotes = inputMode === 'notes' ? notes.trim() : '';

    if (!payloadTopic && !payloadNotes) return;

    setIsLoading(true);
    setErrorState(null);
    setIsFlipped(false);
    setShowHint(false);

    try {
      const res = await fetch('/api/flashcards', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          topic: payloadTopic,
          notes: payloadNotes,
          count,
        }),
      });

      const data = await res.json();
      if (!res.ok) {
        throw new Error(data.error || 'Failed to generate flashcards.');
      }

      setDeck(data);
```

```
    saveStoredFlashcards(data);
    setCurrentIndex(0);
    onDeckUpdated?.();
  } catch (err: any) {
    console.error('Flashcard error:', err);
    const is429 = err.message?.includes('429') || err.message?.includes('Too many requests');
    setErrorState({
      message: err.message || 'Could not generate flashcards. Please try again.',
      isRateLimit: is429,
    });
  } finally {
    setIsLoading(false);
  }
};

const handleToggleKnown = (cardId: string) => {
  if (!deck) return;
  const updatedCards = deck.cards.map((c) =>
    c.id === cardId ? { ...c, known: !c.known } : c
  );
  const updatedDeck = { ...deck, cards: updatedCards };
  setDeck(updatedDeck);
  saveStoredFlashcards(updatedDeck);
  onDeckUpdated?.();
};

const handleShuffle = () => {
  if (!deck) return;
  const shuffled = [...deck.cards].sort(() => Math.random() - 0.5);
  const updatedDeck = { ...deck, cards: shuffled };
  setDeck(updatedDeck);
  setCurrentIndex(0);
  setIsFlipped(false);
  setShowHint(false);
};

// Filter cards based on user selection
const filteredCards = deck
  ? deck.cards.filter((card) => {
      if (filterMode === 'mastered') return card.known === true;
      if (filterMode === 'unmastered') return !card.known;
      return true;
    })
  : [];

const safeIndex = Math.min(currentIndex, Math.max(0, filteredCards.length - 1));
const currentCard = filteredCards[safeIndex];

const handleNext = () => {
  if (safeIndex < filteredCards.length - 1) {
    setIsFlipped(false);
    setShowHint(false);
    setCurrentIndex(safeIndex + 1);
  }
};

const handlePrev = () => {
  if (safeIndex > 0) {
    setIsFlipped(false);
    setShowHint(false);
    setCurrentIndex(safeIndex - 1);
  }
};

const masteredCount = deck ? deck.cards.filter((c) => c.known).length : 0;
const totalDeckCount = deck ? deck.cards.length : 0;

return (
  <div className="max-w-4xl mx-auto space-y-6">
    {/* Flashcard Generation Form */}
    <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 sm:p-8 border border-slate-200/80 dark:border-slate-800/80 shadow-sm tr
      <div className="flex items-center gap-3 mb-5">
        <div className="p-3 rounded-2xl bg-sky-100 dark:bg-sky-950/60 text-sky-600 dark:text-sky-400">
          <Layers className="w-6 h-6" />
        </div>
        <div>
          <h2 className="text-xl font-bold text-slate-900 dark:text-white">
            Flashcard Studio
          </h2>
          <p className="text-sm text-slate-500 dark:text-slate-400">
            Generate active-recall flashcard decks from any topic or raw notes with hints.
          </p>
        </div>
      </div>

      <div className="space-y-4">
        <div className="flex items-center gap-2 border-b border-slate-200 dark:border-slate-800 pb-3">
          <button
            type="button"
```

```
    onClick={() => setInputMode('topic')}
    className={`px-3 py-1.5 rounded-lg text-xs font-semibold transition-all cursor-pointer ${
      inputMode === 'topic'
        ? 'bg-sky-100 dark:bg-sky-950/70 text-sky-700 dark:text-sky-300'
        : 'text-slate-500 hover:text-slate-800 dark:hover:text-slate-200'
    }`}
  >
    Enter Topic
  </button>
  <button
    type="button"
    onClick={() => setInputMode('notes')}
    className={`px-3 py-1.5 rounded-lg text-xs font-semibold transition-all cursor-pointer ${
      inputMode === 'notes'
        ? 'bg-sky-100 dark:bg-sky-950/70 text-sky-700 dark:text-sky-300'
        : 'text-slate-500 hover:text-slate-800 dark:hover:text-slate-200'
    }`}
  >
    From Notes
  </button>
</div>

{inputMode === 'topic' ? (
  <div>
    <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
      Deck Subject or Topic
    </label>
    <input
      type="text"
      value={topic}
      onChange={(e) => setTopic(e.target.value)}
      placeholder="e.g. Spanish Irregular Verbs, Organic Chemistry Functional Groups, World Geography Capitals..."
      className="w-full px-4 py-3 rounded-xl bg-slate-50 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700 te
    />
  </div>
) : (
  <div>
    <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
      Notes to convert into Flashcards
    </label>
    <textarea
      value={notes}
      onChange={(e) => setNotes(e.target.value)}
      rows={4}
      placeholder="Paste key terms, definitions, formulas, or lecture points here..."
      className="w-full px-4 py-3 rounded-xl bg-slate-50 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700 te
    />
  </div>
)}

<div className="flex flex-wrap items-center justify-between gap-3 pt-2">
  <div className="flex items-center gap-2">
    <span className="text-xs text-slate-500 font-medium">Card count:</span>
    <div className="flex gap-1 bg-slate-100 dark:bg-slate-800 p-1 rounded-lg">
      {[6, 8, 12].map((num) => (
        <button
          key={num}
          type="button"
          onClick={() => setCount(num)}
          className={`px-2.5 py-1 text-xs font-semibold rounded-md transition-all cursor-pointer ${
            count === num
              ? 'bg-white dark:bg-slate-700 text-sky-600 dark:text-sky-300 shadow-xs'
              : 'text-slate-500 hover:text-slate-900 dark:hover:text-white'
          }`}
        >
          {num}
        </button>
      ))}
    </div>
  </div>

  <button
    type="button"
    onClick={handleGenerateDeck}
    disabled={(!topic.trim() && !notes.trim()) || isLoading}
    className="px-6 py-2.5 rounded-xl bg-sky-600 hover:bg-sky-700 disabled:opacity-50 text-white font-semibold text-xs shadow-
  >
    {isLoading ? (
      <>
        <Sparkles className="w-3.5 h-3.5 animate-spin" />
        <span>Creating Flashcards...</span>
      </>
    ) : (
      <>
        <Sparkles className="w-3.5 h-3.5" />
        <span>Generate Deck</span>
      </>
    )}
  </button>
```

```
    </div>
  </div>
</div>

{/* Error message */}
{errorState && (
  <ErrorMessage
    message={errorState.message}
    isRateLimit={errorState.isRateLimit}
    onRetry={handleGenerateDeck}
  />
)}

{/* Deck Controls & Active Flashcard */}
{deck && filteredCards.length > 0 && currentCard && (
  <div className="space-y-4 animate-fadeIn">
    {/* Deck Header: Title, Mastered count, Filters */}
    <div className="bg-white dark:bg-slate-900 rounded-2xl p-4 border border-slate-200/80 dark:border-slate-800/80 shadow-xs flex
      <div>
        <h3 className="font-bold text-base text-slate-900 dark:text-white truncate max-w-sm">
          {deck.deckTitle}
        </h3>
        <div className="text-xs text-slate-500 dark:text-slate-400 mt-0.5">
          Mastered: <span className="font-bold text-emerald-600 dark:text-emerald-400">{masteredCount}</span> of {totalDeckCount}
        </div>
      </div>

      <div className="flex items-center gap-2">
        {/* Filter tabs */}
        <div className="flex bg-slate-100 dark:bg-slate-800 p-0.5 rounded-xl text-xs">
          {(['all', 'unmastered', 'mastered'] as const).map((mode) => (
            <button
              key={mode}
              type="button"
              onClick={() => {
                setFilterMode(mode);
                setCurrentIndex(0);
                setIsFlipped(false);
                setShowHint(false);
              }}
              className={`px-2.5 py-1 rounded-lg font-medium capitalize transition-all cursor-pointer ${
                filterMode === mode
                  ? 'bg-white dark:bg-slate-700 text-sky-600 dark:text-sky-300 font-semibold shadow-xs'
                  : 'text-slate-500 hover:text-slate-900 dark:hover:text-white'
              }`}
            >
              {mode}
            </button>
          ))}
        </div>

        <button
          type="button"
          onClick={handleShuffle}
          className="p-2 rounded-xl border border-slate-200 dark:border-slate-700 text-slate-500 hover:text-slate-900 dark:hover:t
          title="Shuffle Deck"
        >
          <Shuffle className="w-4 h-4" />
        </button>
      </div>
    </div>

    {/* 3D Flip Flashcard */}
    <div className="perspective-1000 w-full min-h-[320px] sm:min-h-[360px] flex items-center justify-center">
      <div
        onClick={() => setIsFlipped((prev) => !prev)}
        className={`relative w-full h-[320px] sm:h-[360px] rounded-3xl transition-transform duration-500 transform-style-3d cursor
          isFlipped ? 'rotate-y-180' : ''
        }`}
      >
        {/* FRONT OF CARD */}
        <div className="absolute inset-0 backface-hidden bg-gradient-to-br from-white to-slate-50 dark:from-slate-900 dark:to-slat
          <div className="flex items-center justify-between">
            <span className="text-[11px] font-bold uppercase tracking-wider px-3 py-1 rounded-full bg-sky-100 dark:bg-sky-950 text
              {currentCard.category || 'Concept'}
            </span>
            <div className="flex items-center gap-2 text-xs text-slate-400">
              <span>Card {safeIndex + 1} of {filteredCards.length}</span>
              {currentCard.known && (
                <span className="flex items-center gap-1 text-emerald-600 font-semibold text-xs">
                  <CheckCircle2 className="w-3.5 h-3.5" />
                  Mastered
                </span>
              )}
            </div>
          </div>

          <div className="text-center my-auto px-4">
            <span className="text-xs font-semibold text-sky-600 dark:text-sky-400 uppercase tracking-widest block mb-2">
```

```
            Question / Term
          </span>
          <h4 className="text-xl sm:text-2xl font-bold text-slate-900 dark:text-white leading-relaxed">
            {currentCard.front}
          </h4>

          {/* Hint Toggle */}
          {currentCard.hint && (
            <div className="mt-4" onClick={(e) => e.stopPropagation()}>
              {showHint ? (
                <div className="inline-block px-3 py-1.5 rounded-xl bg-amber-50 dark:bg-amber-950/40 text-amber-700 dark:text-am
                  ■ Hint: {currentCard.hint}
                </div>
              ) : (
                <button
                  type="button"
                  onClick={() => setShowHint(true)}
                  className="inline-flex items-center gap-1 text-xs text-slate-400 hover:text-amber-500 transition-colors cursor
                >
                  <Eye className="w-3.5 h-3.5" />
                  <span>Show hint</span>
                </button>
              )}
            </div>
          )}
        </div>

        <div className="flex items-center justify-between text-xs text-slate-400 dark:text-slate-500 pt-2 border-t border-slate-
          <span>Click or press Space to reveal answer</span>
          <RotateCw className="w-4 h-4 animate-spin-slow" />
        </div>
      </div>

      {/* BACK OF CARD */}
      <div className="absolute inset-0 backface-hidden rotate-y-180 bg-gradient-to-br from-indigo-900 to-slate-900 text-white ro
        <div className="flex items-center justify-between">
          <span className="text-[11px] font-bold uppercase tracking-wider px-3 py-1 rounded-full bg-indigo-800/80 text-indigo-20
            Answer & Explanation
          </span>
          <span className="text-xs text-indigo-300">
            Card {safeIndex + 1} of {filteredCards.length}
          </span>
        </div>

        <div className="my-auto px-4 text-center">
          <p className="text-base sm:text-xl font-medium leading-relaxed text-indigo-100">
            {currentCard.back}
          </p>
        </div>

        <div className="flex items-center justify-between text-xs text-indigo-300 pt-2 border-t border-indigo-800/60">
          <span>Click to flip back</span>
          <RotateCw className="w-4 h-4" />
        </div>
      </div>
    </div>
  </div>

  {/* Navigation & Mastery Controls */}
  <div className="flex flex-wrap items-center justify-between gap-3 pt-2">
    <div className="flex items-center gap-2">
      <button
        type="button"
        onClick={handlePrev}
        disabled={safeIndex === 0}
        className="p-2.5 rounded-xl border border-slate-200 dark:border-slate-700 bg-white dark:bg-slate-900 text-slate-700 dark
        title="Previous card (Left Arrow)"
      >
        <ChevronLeft className="w-5 h-5" />
      </button>

      <button
        type="button"
        onClick={handleNext}
        disabled={safeIndex === filteredCards.length - 1}
        className="p-2.5 rounded-xl border border-slate-200 dark:border-slate-700 bg-white dark:bg-slate-900 text-slate-700 dark
        title="Next card (Right Arrow)"
      >
        <ChevronRight className="w-5 h-5" />
      </button>

      <span className="text-xs text-slate-400 font-medium px-2">
        Card {safeIndex + 1} / {filteredCards.length}
      </span>
    </div>

    {/* "Mark as Known" Toggle */}
    <button
      type="button"
```

```
              onClick={() => handleToggleKnown(currentCard.id)}
              className={`px-4 py-2.5 rounded-xl text-xs font-semibold flex items-center gap-2 transition-all cursor-pointer ${
                currentCard.known
                  ? 'bg-emerald-100 dark:bg-emerald-950/60 text-emerald-800 dark:text-emerald-200 border border-emerald-300 dark:border-
                  : 'bg-white dark:bg-slate-900 text-slate-700 dark:text-slate-300 border border-slate-200 dark:border-slate-700 hover:b
              }`}
            >
              <CheckCircle2
                className={`w-4 h-4 ${
                  currentCard.known ? 'text-emerald-600 dark:text-emerald-400' : 'text-slate-400'
                }`}
              />
              <span>{currentCard.known ? 'Marked as Known' : 'Mark as Known'}</span>
            </button>
          </div>
        </div>
      )}

      {/* If filtered cards is empty */}
      {deck && filteredCards.length === 0 && (
        <div className="p-8 text-center bg-white dark:bg-slate-900 rounded-3xl border border-slate-200 dark:border-slate-800">
          <BookmarkCheck className="w-10 h-10 text-emerald-500 mx-auto mb-2" />
          <h4 className="font-bold text-base text-slate-900 dark:text-white">
            No cards found in this filter
          </h4>
          <p className="text-xs text-slate-500 mt-1 mb-4">
            Try switching back to "All" to review your deck.
          </p>
          <button
            type="button"
            onClick={() => setFilterMode('all')}
            className="px-4 py-2 rounded-xl bg-sky-600 text-white text-xs font-semibold cursor-pointer"
          >
            Show All Cards
          </button>
        </div>
      )}
    </div>
  );
};
```

src/components/planner/StudyPlanner.tsx

```
import React, { useState } from 'react';
import {
  CalendarDays,
  Sparkles,
  Clock,
  BookOpen,
  CheckCircle2,
  Printer,
  RotateCcw,
  Calendar,
  AlertTriangle,
  Award,
  Zap,
} from 'lucide-react';
import { StudyPlan, StudyDay, StudyBlock } from '../../types';
import { ErrorMessage } from '../common/ErrorMessage';
import { getStoredStudyPlan, saveStoredStudyPlan } from '../../utils/storage';

export const StudyPlanner: React.FC = () => {
  const [subjectsInput, setSubjectsInput] = useState('');
  const [examDate, setExamDate] = useState('');
  const [hoursPerDay, setHoursPerDay] = useState<number>(3);
  const [pace, setPace] = useState<'Light' | 'Balanced' | 'Intensive'>('Balanced');

  const [isLoading, setIsLoading] = useState(false);
  const [plan, setPlan] = useState<StudyPlan | null>(() => getStoredStudyPlan());
  const [errorState, setErrorState] = useState<{ message: string; isRateLimit: boolean } | null>(null);
  const [viewMode, setViewMode] = useState<'timeline' | 'table'>('timeline');

  const handleGeneratePlan = async () => {
    if (!subjectsInput.trim()) return;

    setIsLoading(true);
    setErrorState(null);

    try {
      const res = await fetch('/api/planner', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          subjects: subjectsInput.split(',').map((s) => s.trim()).filter(Boolean),
          examDate: examDate || 'In 14 days',
          hoursPerDay,
          pace,
        }),
      });

      const data = await res.json();
      if (!res.ok) {
        throw new Error(data.error || 'Failed to generate study plan.');
      }

      const newPlan: StudyPlan = {
        ...data,
        createdDate: new Date().toLocaleDateString(),
      };

      setPlan(newPlan);
      saveStoredStudyPlan(newPlan);
    } catch (err: any) {
      console.error('Planner error:', err);
      const is429 = err.message?.includes('429') || err.message?.includes('Too many requests');
      setErrorState({
        message: err.message || 'Could not generate study schedule. Please try again.',
        isRateLimit: is429,
      });
    } finally {
      setIsLoading(false);
    }
  };

  const handleToggleBlock = (dayIndex: number, blockIndex: number) => {
    if (!plan) return;
    const updatedDays = [...plan.days];
    const targetBlock = updatedDays[dayIndex].blocks[blockIndex];
    targetBlock.completed = !targetBlock.completed;

    const updatedPlan = { ...plan, days: updatedDays };
    setPlan(updatedPlan);
    saveStoredStudyPlan(updatedPlan);
  };

  const handlePrint = () => {
    window.print();
  };

  // Calculate overall completed blocks percentage
```

```
const totalBlocks = plan
  ? plan.days.reduce((acc, d) => acc + d.blocks.length, 0)
  : 0;
const completedBlocks = plan
  ? plan.days.reduce(
      (acc, d) => acc + d.blocks.filter((b) => b.completed).length,
      0
    )
  : 0;
const completionPercentage = totalBlocks > 0 ? Math.round((completedBlocks / totalBlocks) * 100) : 0;

return (
  <div className="max-w-5xl mx-auto space-y-6">
    {/* Planner Generator Form */}
    <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 sm:p-8 border border-slate-200/80 dark:border-slate-800/80 shadow-sm tr
      <div className="flex items-center gap-3 mb-5">
        <div className="p-3 rounded-2xl bg-emerald-100 dark:bg-emerald-950/60 text-emerald-600 dark:text-emerald-400">
          <CalendarDays className="w-6 h-6" />
        </div>
        <div>
          <h2 className="text-xl font-bold text-slate-900 dark:text-white">
            Smart Study Planner
          </h2>
          <p className="text-sm text-slate-500 dark:text-slate-400">
            Create a personalized daily study timeline with active recall and scheduled revision cycles.
          </p>
        </div>
      </div>

      <form
        onSubmit={(e) => {
          e.preventDefault();
          handleGeneratePlan();
        }}
        className="space-y-4"
      >
        <div>
          <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
            Subjects / Topics to Cover (separated by commas)
          </label>
          <input
            type="text"
            value={subjectsInput}
            onChange={(e) => setSubjectsInput(e.target.value)}
            placeholder="e.g. Organic Chemistry, Linear Algebra, Macroeconomics, Modern European History"
            className="w-full px-4 py-3 rounded-xl bg-slate-50 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700 text
          />
        </div>

        <div className="grid grid-cols-1 sm:grid-cols-3 gap-4">
          <div>
            <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
              Target Exam Date or Deadline
            </label>
            <input
              type="date"
              value={examDate}
              onChange={(e) => setExamDate(e.target.value)}
              className="w-full px-4 py-2.5 rounded-xl bg-slate-50 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700
            />
          </div>

          <div>
            <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
              Available Study Hours / Day: <strong className="text-emerald-600">{hoursPerDay} hrs</strong>
            </label>
            <input
              type="range"
              min="1"
              max="8"
              step="0.5"
              value={hoursPerDay}
              onChange={(e) => setHoursPerDay(parseFloat(e.target.value))}
              className="w-full accent-emerald-600 mt-2 cursor-pointer"
            />
          </div>

          <div>
            <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
              Study Pace
            </label>
            <div className="grid grid-cols-3 gap-1 bg-slate-100 dark:bg-slate-800 p-1 rounded-xl">
              {(['Light', 'Balanced', 'Intensive'] as const).map((p) => (
                <button
                  key={p}
                  type="button"
                  onClick={() => setPace(p)}
                  className={`py-2 text-[11px] font-semibold rounded-lg transition-all cursor-pointer ${
                    pace === p
```

```
                ? 'bg-white dark:bg-slate-700 text-emerald-600 dark:text-emerald-300 shadow-xs'
                : 'text-slate-600 dark:text-slate-400'
            }`}
          >
            {p}
          </button>
        ))}
      </div>
    </div>
  </div>

  <div className="pt-2 flex justify-end">
    <button
      type="submit"
      disabled={!subjectsInput.trim() || isLoading}
      className="px-6 py-3 rounded-xl bg-emerald-600 hover:bg-emerald-700 disabled:opacity-50 text-white font-semibold text-sm s
    >
      {isLoading ? (
        <>
          <Sparkles className="w-4 h-4 animate-spin" />
          <span>Structuring Study Roadmap...</span>
        </>
      ) : (
        <>
          <Sparkles className="w-4 h-4" />
          <span>Generate Schedule</span>
        </>
      )}
    </button>
  </div>
 </form>
</div>

{/* Error display */}
{errorState && (
 <ErrorMessage
   message={errorState.message}
   isRateLimit={errorState.isRateLimit}
   onRetry={handleGeneratePlan}
 />
)}

{/* Generated Study Schedule */}
{plan && (
 <div className="space-y-4 animate-fadeIn">
   {/* Plan Header Card */}
   <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 sm:p-8 border border-slate-200/80 dark:border-slate-800/80 shadow-s
     <div className="space-y-1">
       <div className="flex items-center gap-2">
         <span className="px-2.5 py-0.5 rounded-full text-xs font-bold bg-emerald-100 dark:bg-emerald-950/60 text-emerald-700 dar
           {plan.totalDays} Days Roadmap
         </span>
         <span className="text-xs text-slate-400">Target: {plan.dailyHoursTarget} hrs/day</span>
       </div>
       <h3 className="text-xl sm:text-2xl font-black text-slate-900 dark:text-white">
         {plan.planTitle}
       </h3>
       <p className="text-xs sm:text-sm text-slate-500 dark:text-slate-400 max-w-2xl leading-relaxed">
         {plan.overview}
       </p>
     </div>

     {/* Print and View Toggle */}
     <div className="flex items-center gap-2 shrink-0 print:hidden">
       <div className="flex bg-slate-100 dark:bg-slate-800 p-1 rounded-xl text-xs">
         <button
           type="button"
           onClick={() => setViewMode('timeline')}
           className={`px-3 py-1.5 rounded-lg font-medium transition-all cursor-pointer ${
             viewMode === 'timeline'
               ? 'bg-white dark:bg-slate-700 text-emerald-600 dark:text-emerald-300 font-semibold shadow-xs'
               : 'text-slate-500 hover:text-slate-900 dark:hover:text-white'
           }`}
         >
           Timeline
         </button>
         <button
           type="button"
           onClick={() => setViewMode('table')}
           className={`px-3 py-1.5 rounded-lg font-medium transition-all cursor-pointer ${
             viewMode === 'table'
               ? 'bg-white dark:bg-slate-700 text-emerald-600 dark:text-emerald-300 font-semibold shadow-xs'
               : 'text-slate-500 hover:text-slate-900 dark:hover:text-white'
           }`}
         >
           Table
         </button>
       </div>

       <button
```

```
      type="button"
      onClick={handlePrint}
      className="p-2.5 rounded-xl border border-slate-200 dark:border-slate-700 text-slate-600 dark:text-slate-300 hover:bg-sl
      title="Print or Save PDF"
    >
      <Printer className="w-4 h-4" />
    </button>
  </div>
</div>

{/* Progress Tracker */}
<div className="bg-white dark:bg-slate-900 rounded-2xl p-4 border border-slate-200/80 dark:border-slate-800/80 shadow-xs flex
  <div className="flex items-center gap-3">
    <div className="w-10 h-10 rounded-xl bg-emerald-100 dark:bg-emerald-950/60 text-emerald-600 dark:text-emerald-400 flex ite
      {completionPercentage}%
    </div>
    <div>
      <div className="text-xs font-bold text-slate-900 dark:text-white">
        Study Progress: {completedBlocks} of {totalBlocks} sessions completed
      </div>
      <div className="w-48 sm:w-72 h-1.5 bg-slate-100 dark:bg-slate-800 rounded-full mt-1.5 overflow-hidden">
        <div
          className="h-full bg-emerald-500 transition-all duration-300"
          style={{ width: `${completionPercentage}%` }}
        />
      </div>
    </div>
  </div>

  {completionPercentage === 100 && (
    <span className="text-xs font-bold text-emerald-600 flex items-center gap-1 animate-bounce">
      <Award className="w-4 h-4" />
      All Sessions Completed!
    </span>
  )}
</div>

{/* Day by Day View */}
{viewMode === 'timeline' ? (
  <div className="space-y-4">
    {plan.days.map((day, dayIdx) => (
      <div
        key={day.dayNumber}
        className={`rounded-3xl p-6 border transition-all ${
          day.isRevisionDay
            ? 'bg-gradient-to-r from-purple-50/50 to-indigo-50/30 dark:from-purple-950/20 dark:to-indigo-950/20 border-purple-
            : 'bg-white dark:bg-slate-900 border-slate-200/80 dark:border-slate-800/80 shadow-xs'
        }`}
      >
        {/* Day Header */}
        <div className="flex flex-wrap items-center justify-between gap-2 mb-3 pb-2 border-b border-slate-100 dark:border-slat
          <div className="flex items-center gap-2.5">
            <span className="font-extrabold text-sm sm:text-base text-slate-900 dark:text-white">
              {day.dateOrLabel}
            </span>
            {day.isRevisionDay && (
              <span className="px-2 py-0.5 rounded-full text-[10px] font-bold bg-purple-100 dark:bg-purple-950 text-purple-700
                ■ Spaced Revision Day
              </span>
            )}
          </div>
          <span className="text-xs font-semibold text-slate-500 dark:text-slate-400">
            Focus: <strong className="text-slate-800 dark:text-slate-200">{day.theme}</strong>
          </span>
        </div>

        {/* Study Blocks */}
        <div className="grid grid-cols-1 md:grid-cols-2 gap-3 mb-3">
          {day.blocks.map((block, blockIdx) => (
            <div
              key={blockIdx}
              onClick={() => handleToggleBlock(dayIdx, blockIdx)}
              className={`p-3.5 rounded-2xl border text-xs sm:text-sm flex items-start gap-3 transition-all cursor-pointer ${
                block.completed
                  ? 'bg-emerald-50/60 dark:bg-emerald-950/30 border-emerald-300 dark:border-emerald-800 text-slate-500 dark:te
                  : 'bg-slate-50/80 dark:bg-slate-800/50 border-slate-200 dark:border-slate-700/80 text-slate-800 dark:text-sl
              }`}
            >
              <div className="mt-0.5 shrink-0">
                {block.completed ? (
                  <CheckCircle2 className="w-5 h-5 text-emerald-500" />
                ) : (
                  <div className="w-5 h-5 rounded-full border-2 border-slate-300 dark:border-slate-600 hover:border-emerald-50
                )}
              </div>

              <div className="flex-1 min-w-0">
                <div className="flex items-center justify-between gap-1 mb-1">
                  <span className="font-bold text-xs text-indigo-600 dark:text-indigo-400">
```

```
                          {block.timeSlot}
                        </span>
                        <span className="text-[10px] font-semibold px-2 py-0.5 rounded-md bg-slate-200/80 dark:bg-slate-700 text-sla
                          {block.technique}
                        </span>
                      </div>
                      <div className={`font-semibold text-xs text-slate-900 dark:text-white ${block.completed ? 'line-through opacit
                        {block.subject}
                      </div>
                      <div className="text-xs text-slate-600 dark:text-slate-300 mt-0.5 leading-snug">
                        {block.task}
                      </div>
                    </div>
                  </div>
                ))}
              </div>

              {/* Daily Retention Tip */}
              {day.dailyTip && (
                <div className="text-[11px] text-slate-500 dark:text-slate-400 italic flex items-center gap-1.5 pt-1">
                  <Zap className="w-3.5 h-3.5 text-amber-500 shrink-0" />
                  <span>{day.dailyTip}</span>
                </div>
              )}
            </div>
          ))}
        </div>
      ) : (
        /* Table View */
        <div className="overflow-x-auto rounded-3xl border border-slate-200/80 dark:border-slate-800/80 bg-white dark:bg-slate-900 s
          <table className="w-full text-left text-xs divide-y divide-slate-200 dark:divide-slate-800">
            <thead className="bg-slate-50 dark:bg-slate-800/70 font-semibold text-slate-700 dark:text-slate-300">
              <tr>
                <th className="px-4 py-3">Day / Date</th>
                <th className="px-4 py-3">Focus Theme</th>
                <th className="px-4 py-3">Sessions & Tasks</th>
                <th className="px-4 py-3">Study Technique</th>
              </tr>
            </thead>
            <tbody className="divide-y divide-slate-100 dark:divide-slate-800/50">
              {plan.days.map((day) => (
                <tr
                  key={day.dayNumber}
                  className={day.isRevisionDay ? 'bg-purple-50/30 dark:bg-purple-950/20 font-medium' : ''}
                >
                  <td className="px-4 py-3 font-bold text-slate-900 dark:text-white whitespace-nowrap">
                    {day.dateOrLabel}
                    {day.isRevisionDay && (
                      <span className="block text-[10px] text-purple-600 font-semibold">Revision</span>
                    )}
                  </td>
                  <td className="px-4 py-3 text-slate-700 dark:text-slate-300">
                    {day.theme}
                  </td>
                  <td className="px-4 py-3 space-y-1">
                    {day.blocks.map((b, idx) => (
                      <div key={idx} className="leading-snug">
                        <strong className="text-indigo-600 dark:text-indigo-400">[{b.timeSlot} - {b.subject}]</strong> {b.task}
                      </div>
                    ))}
                  </td>
                  <td className="px-4 py-3 text-slate-500 dark:text-slate-400 whitespace-nowrap">
                    {day.blocks.map((b) => b.technique).join(', ')}
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </div>
  );
};
```

src/components/quiz/QuizGenerator.tsx

```
import React, { useState } from 'react';
import confetti from 'canvas-confetti';
import {
  HelpCircle,
  Sparkles,
  CheckCircle2,
  XCircle,
  RotateCcw,
  Trophy,
  ArrowRight,
  Clock,
  History,
  BookOpen,
} from 'lucide-react';
import { QuizData, QuizResultRecord } from '../../types';
import { ErrorMessage } from '../common/ErrorMessage';
import { saveQuizResult, getStoredQuizHistory } from '../../utils/storage';

interface QuizGeneratorProps {
  onQuizCompleted?: () => void;
}

export const QuizGenerator: React.FC<QuizGeneratorProps> = ({ onQuizCompleted }) => {
  const [topic, setTopic] = useState('');
  const [notes, setNotes] = useState('');
  const [inputMode, setInputMode] = useState<'topic' | 'notes'>('topic');
  const [questionCount, setQuestionCount] = useState<5 | 10>(5);
  const [difficulty, setDifficulty] = useState<'Easy' | 'Medium' | 'Hard'>('Medium');

  const [isLoading, setIsLoading] = useState(false);
  const [quizData, setQuizData] = useState<QuizData | null>(null);
  const [currentQuestionIndex, setCurrentQuestionIndex] = useState(0);
  const [selectedAnswers, setSelectedAnswers] = useState<Record<number, number>>({});
  const [showInstantFeedback, setShowInstantFeedback] = useState(true);
  const [isCompleted, setIsCompleted] = useState(false);
  const [errorState, setErrorState] = useState<{ message: string; isRateLimit: boolean } | null>(null);

  // Past quiz history
  const [quizHistory, setQuizHistory] = useState<QuizResultRecord[]>(() => getStoredQuizHistory());

  const handleGenerateQuiz = async () => {
    const payloadTopic = inputMode === 'topic' ? topic.trim() : '';
    const payloadNotes = inputMode === 'notes' ? notes.trim() : '';

    if (!payloadTopic && !payloadNotes) return;

    setIsLoading(true);
    setErrorState(null);
    setSelectedAnswers({});
    setCurrentQuestionIndex(0);
    setIsCompleted(false);

    try {
      const res = await fetch('/api/quiz', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          topic: payloadTopic,
          notes: payloadNotes,
          count: questionCount,
          difficulty,
        }),
      });

      const data = await res.json();
      if (!res.ok) {
        throw new Error(data.error || 'Failed to generate quiz.');
      }

      setQuizData(data);
    } catch (err: any) {
      console.error('Quiz error:', err);
      const is429 = err.message?.includes('429') || err.message?.includes('Too many requests');
      setErrorState({
        message: err.message || 'Could not generate quiz. Please try again.',
        isRateLimit: is429,
      });
    } finally {
      setIsLoading(false);
    }
  };

  const handleSelectOption = (questionIndex: number, optionIndex: number) => {
    // Only allow selecting once per question
    if (selectedAnswers[questionIndex] !== undefined) return;

    const newAnswers = { ...selectedAnswers, [questionIndex]: optionIndex };
```

```
  setSelectedAnswers(newAnswers);

  // If it's the last question, or if all are answered, check completion
  const allAnswered = quizData && Object.keys(newAnswers).length === quizData.questions.length;
  if (allAnswered) {
    finishQuiz(newAnswers);
  }
};

const finishQuiz = (finalAnswers: Record<number, number>) => {
  if (!quizData) return;

  let score = 0;
  quizData.questions.forEach((q, idx) => {
    if (finalAnswers[idx] === q.correctIndex) {
      score++;
    }
  });

  const percentage = Math.round((score / quizData.questions.length) * 100);

  // Trigger confetti if high score
  if (percentage >= 60) {
    try {
      confetti({
        particleCount: 100,
        spread: 70,
        origin: { y: 0.6 },
      });
    } catch {}
  }

  const record: QuizResultRecord = {
    id: `quiz-${Date.now()}`,
    topic: quizData.topic || topic || 'Custom Practice',
    score,
    total: quizData.questions.length,
    percentage,
    date: new Date().toLocaleDateString(),
    difficulty: quizData.difficulty,
  };

  saveQuizResult(record);
  setQuizHistory(getStoredQuizHistory());
  setIsCompleted(true);
  onQuizCompleted?.();
};

const handleRetry = () => {
  setSelectedAnswers({});
  setCurrentQuestionIndex(0);
  setIsCompleted(false);
};

const currentQ = quizData?.questions[currentQuestionIndex];
const userSelected = currentQ ? selectedAnswers[currentQuestionIndex] : undefined;
const isAnswered = userSelected !== undefined;

// Calculate score so far
const totalCorrect = quizData
  ? quizData.questions.reduce((acc, q, idx) => {
      return selectedAnswers[idx] === q.correctIndex ? acc + 1 : acc;
    }, 0)
  : 0;

return (
  <div className="max-w-4xl mx-auto space-y-6">
    {/* Quiz Creator Form */}
    {!quizData && (
      <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 sm:p-8 border border-slate-200/80 dark:border-slate-800/80 shadow-sm
        <div className="flex items-center gap-3 mb-6">
          <div className="p-3 rounded-2xl bg-purple-100 dark:bg-purple-950/60 text-purple-600 dark:text-purple-400">
            <HelpCircle className="w-6 h-6" />
          </div>
          <div>
            <h2 className="text-xl font-bold text-slate-900 dark:text-white">
              Interactive Quiz Generator
            </h2>
            <p className="text-sm text-slate-500 dark:text-slate-400">
              Generate tailored multiple-choice tests with step-by-step answer rationales.
            </p>
          </div>
        </div>

        <div className="space-y-4">
          {/* Input Mode Toggle */}
          <div className="flex items-center gap-2 border-b border-slate-200 dark:border-slate-800 pb-3">
            <button
              type="button"
```

```
    onClick={() => setInputMode('topic')}
    className={`px-3 py-1.5 rounded-lg text-xs font-semibold transition-all cursor-pointer ${
      inputMode === 'topic'
        ? 'bg-purple-100 dark:bg-purple-950/70 text-purple-700 dark:text-purple-300'
        : 'text-slate-500 hover:text-slate-800 dark:hover:text-slate-200'
    }`}
  >
    Enter Topic
  </button>
  <button
    type="button"
    onClick={() => setInputMode('notes')}
    className={`px-3 py-1.5 rounded-lg text-xs font-semibold transition-all cursor-pointer ${
      inputMode === 'notes'
        ? 'bg-purple-100 dark:bg-purple-950/70 text-purple-700 dark:text-purple-300'
        : 'text-slate-500 hover:text-slate-800 dark:hover:text-slate-200'
    }`}
  >
    Paste Study Notes
  </button>
</div>

{inputMode === 'topic' ? (
  <div>
    <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
      Quiz Topic
    </label>
    <input
      type="text"
      value={topic}
      onChange={(e) => setTopic(e.target.value)}
      placeholder="e.g. World War II European Theater, Organic Chemistry Isomers, Calculus Integrals..."
      className="w-full px-4 py-3 rounded-xl bg-slate-50 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700
    />
  </div>
) : (
  <div>
    <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
      Notes Content
    </label>
    <textarea
      value={notes}
      onChange={(e) => setNotes(e.target.value)}
      rows={5}
      placeholder="Paste lecture text, textbook summary, or concepts to generate questions from..."
      className="w-full px-4 py-3 rounded-xl bg-slate-50 dark:bg-slate-800/80 border border-slate-200 dark:border-slate-700
    />
  </div>
)}

{/* Questions count & Difficulty */}
<div className="grid grid-cols-1 sm:grid-cols-2 gap-4 pt-2">
  <div>
    <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
      Number of Questions
    </label>
    <div className="grid grid-cols-2 gap-2 bg-slate-100 dark:bg-slate-800 p-1 rounded-xl">
      {([5, 10] as const).map((cnt) => (
        <button
          key={cnt}
          type="button"
          onClick={() => setQuestionCount(cnt)}
          className={`py-2 text-xs font-semibold rounded-lg transition-all cursor-pointer ${
            questionCount === cnt
              ? 'bg-white dark:bg-slate-700 text-purple-600 dark:text-purple-300 shadow-xs'
              : 'text-slate-600 dark:text-slate-400'
          }`}
        >
          {cnt} Questions
        </button>
      ))}
    </div>
  </div>

  <div>
    <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-1.5">
      Difficulty Level
    </label>
    <div className="grid grid-cols-3 gap-1.5 bg-slate-100 dark:bg-slate-800 p-1 rounded-xl">
      {(['Easy', 'Medium', 'Hard'] as const).map((diff) => (
        <button
          key={diff}
          type="button"
          onClick={() => setDifficulty(diff)}
          className={`py-2 text-xs font-semibold rounded-lg transition-all cursor-pointer ${
            difficulty === diff
              ? 'bg-white dark:bg-slate-700 text-purple-600 dark:text-purple-300 shadow-xs'
              : 'text-slate-600 dark:text-slate-400'
          }`}
```

```
              >
                {diff}
              </button>
            ))}
          </div>
        </div>
      </div>

      <div className="pt-4 flex justify-end">
        <button
          type="button"
          onClick={handleGenerateQuiz}
          disabled={(!topic.trim() && !notes.trim()) || isLoading}
          className="px-6 py-3 rounded-xl bg-purple-600 hover:bg-purple-700 disabled:opacity-50 text-white font-semibold text-sm s
        >
          {isLoading ? (
            <>
              <Sparkles className="w-4 h-4 animate-spin" />
              <span>Formulating Questions...</span>
            </>
          ) : (
            <>
              <Sparkles className="w-4 h-4" />
              <span>Generate Quiz</span>
            </>
          )}
        </button>
      </div>
    </div>
  </div>
)}

{/* Error state */}
{errorState && (
  <ErrorMessage
    message={errorState.message}
    isRateLimit={errorState.isRateLimit}
    onRetry={handleGenerateQuiz}
  />
)}

{/* Active Quiz Gameplay */}
{quizData && !isCompleted && currentQ && (
  <div className="space-y-4 animate-fadeIn">
    {/* Quiz Top bar: Progress, Question count, reset */}
    <div className="bg-white dark:bg-slate-900 rounded-2xl p-4 border border-slate-200/80 dark:border-slate-800/80 shadow-xs flex
      <div>
        <div className="text-xs font-bold uppercase tracking-wider text-purple-600 dark:text-purple-400">
          {quizData.topic || topic} • {quizData.difficulty}
        </div>
        <div className="text-sm font-semibold text-slate-900 dark:text-white">
          Question {currentQuestionIndex + 1} of {quizData.questions.length}
        </div>
      </div>

      {/* Score so far badge */}
      <div className="flex items-center gap-3">
        <div className="text-xs font-medium text-slate-500 dark:text-slate-400">
          Score: <span className="font-bold text-indigo-600 dark:text-indigo-400">{totalCorrect}</span> / {Object.keys(selectedAns
        </div>

        <button
          type="button"
          onClick={() => setQuizData(null)}
          className="text-xs font-medium text-slate-500 hover:text-rose-500 dark:text-slate-400 transition-colors cursor-pointer"
        >
          Exit Quiz
        </button>
      </div>
    </div>

    {/* Progress bar */}
    <div className="w-full h-2 bg-slate-200 dark:bg-slate-800 rounded-full overflow-hidden">
      <div
        className="h-full bg-gradient-to-r from-purple-600 to-indigo-600 transition-all duration-300"
        style={{
          width: `${((currentQuestionIndex + 1) / quizData.questions.length) * 100}%`,
        }}
      />
    </div>

    {/* Question Card */}
    <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 sm:p-8 border border-slate-200/80 dark:border-slate-800/80 shadow-s
      <h3 className="text-lg sm:text-xl font-bold text-slate-900 dark:text-white leading-relaxed">
        {currentQ.question}
      </h3>

      {/* Options List */}
      <div className="space-y-3">
```

```
  {currentQ.options.map((option, optIdx) => {
    const isSelected = userSelected === optIdx;
    const isCorrect = optIdx === currentQ.correctIndex;

    let stateClasses = 'border-slate-200 dark:border-slate-700 hover:border-purple-300 bg-slate-50/70 dark:bg-slate-800/50';

    if (isAnswered) {
      if (isCorrect) {
        stateClasses = 'border-emerald-500 bg-emerald-50 dark:bg-emerald-950/40 text-emerald-900 dark:text-emerald-200 font-
      } else if (isSelected) {
        stateClasses = 'border-rose-500 bg-rose-50 dark:bg-rose-950/40 text-rose-900 dark:text-rose-200 ring-1 ring-rose-500
      } else {
        stateClasses = 'opacity-60 border-slate-200 dark:border-slate-800';
      }
    }

    return (
      <button
        key={optIdx}
        type="button"
        disabled={isAnswered}
        onClick={() => handleSelectOption(currentQuestionIndex, optIdx)}
        className={`w-full text-left p-4 rounded-2xl border text-sm transition-all flex items-center justify-between cursor-
      >
        <div className="flex items-center gap-3">
          <span className="w-7 h-7 rounded-xl bg-slate-200 dark:bg-slate-700 flex items-center justify-center font-bold text
            {String.fromCharCode(65 + optIdx)}
          </span>
          <span className="leading-snug">{option}</span>
        </div>

        {isAnswered && isCorrect && (
          <CheckCircle2 className="w-5 h-5 text-emerald-500 shrink-0 ml-2" />
        )}
        {isAnswered && isSelected && !isCorrect && (
          <XCircle className="w-5 h-5 text-rose-500 shrink-0 ml-2" />
        )}
      </button>
    );
  })}
</div>

{/* Instant Feedback Card */}
{isAnswered && (
  <div className="p-4 sm:p-5 rounded-2xl bg-indigo-50/70 dark:bg-indigo-950/40 border border-indigo-200/80 dark:border-indig
    <div className="flex items-center gap-2 font-bold text-sm">
      {userSelected === currentQ.correctIndex ? (
        <span className="text-emerald-600 dark:text-emerald-400 flex items-center gap-1.5">
          <CheckCircle2 className="w-4 h-4" /> Correct! Outstanding!
        </span>
      ) : (
        <span className="text-rose-600 dark:text-rose-400 flex items-center gap-1.5">
          <XCircle className="w-4 h-4" /> Incorrect
        </span>
      )}
    </div>
    <p className="text-xs sm:text-sm text-slate-700 dark:text-slate-300 leading-relaxed">
      {currentQ.explanation}
    </p>
  </div>
)}

{/* Navigation buttons */}
<div className="flex items-center justify-between pt-2">
  <button
    type="button"
    disabled={currentQuestionIndex === 0}
    onClick={() => setCurrentQuestionIndex((prev) => prev - 1)}
    className="px-4 py-2 rounded-xl border border-slate-200 dark:border-slate-700 text-xs font-semibold text-slate-600 dark:
  >
    Previous
  </button>

  {currentQuestionIndex < quizData.questions.length - 1 ? (
    <button
      type="button"
      onClick={() => setCurrentQuestionIndex((prev) => prev + 1)}
      disabled={!isAnswered}
      className="px-5 py-2.5 rounded-xl bg-purple-600 hover:bg-purple-700 disabled:opacity-40 text-white font-semibold text-
    >
      <span>Next Question</span>
      <ArrowRight className="w-3.5 h-3.5" />
    </button>
  ) : (
    <button
      type="button"
      onClick={() => finishQuiz(selectedAnswers)}
      disabled={!isAnswered}
      className="px-6 py-2.5 rounded-xl bg-emerald-600 hover:bg-emerald-700 disabled:opacity-40 text-white font-semibold tex
```

```
          >
            <span>View Final Results</span>
            <Trophy className="w-3.5 h-3.5" />
          </button>
        )}
      </div>
    </div>
  </div>
)}

{/* Completion & Final Score Screen */}
{quizData && isCompleted && (
  <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 sm:p-10 border border-slate-200/80 dark:border-slate-800/80 shadow-mo
    <div className="w-20 h-20 rounded-3xl bg-gradient-to-tr from-amber-400 via-purple-500 to-indigo-600 flex items-center justify-
      <Trophy className="w-10 h-10" />
    </div>

    <div>
      <h3 className="text-2xl sm:text-3xl font-extrabold text-slate-900 dark:text-white">
        Quiz Completed!
      </h3>
      <p className="text-sm text-slate-500 dark:text-slate-400 mt-1">
        {quizData.topic} • {quizData.difficulty} Difficulty
      </p>
    </div>

    {/* Score Badge */}
    <div className="inline-block p-6 rounded-3xl bg-slate-50 dark:bg-slate-800/60 border border-slate-200 dark:border-slate-700">
      <div className="text-4xl sm:text-5xl font-black text-indigo-600 dark:text-indigo-400">
        {totalCorrect} / {quizData.questions.length}
      </div>
      <div className="text-sm font-semibold text-slate-600 dark:text-slate-300 mt-1">
        Score: {Math.round((totalCorrect / quizData.questions.length) * 100)}%
      </div>
    </div>

    {/* Action buttons */}
    <div className="flex flex-wrap items-center justify-center gap-3 pt-2">
      <button
        type="button"
        onClick={handleRetry}
        className="px-5 py-2.5 rounded-xl bg-purple-600 hover:bg-purple-700 text-white font-semibold text-xs flex items-center gap
      >
        <RotateCcw className="w-3.5 h-3.5" />
        <span>Retry This Quiz</span>
      </button>

      <button
        type="button"
        onClick={() => {
          setQuizData(null);
          setTopic('');
          setNotes('');
        }}
        className="px-5 py-2.5 rounded-xl border border-slate-200 dark:border-slate-700 text-slate-700 dark:text-slate-300 hover:b
      >
        New Quiz Topic
      </button>
    </div>

    {/* Review Detailed Answers Breakdown */}
    <div className="pt-6 border-t border-slate-200 dark:border-slate-800 text-left space-y-4">
      <h4 className="font-bold text-sm text-slate-900 dark:text-white">
        Question Review & Explanations:
      </h4>
      <div className="space-y-4">
        {quizData.questions.map((q, idx) => {
          const userAns = selectedAnswers[idx];
          const isCorrect = userAns === q.correctIndex;
          return (
            <div
              key={idx}
              className={`p-4 rounded-2xl border text-xs sm:text-sm ${
                isCorrect
                  ? 'border-emerald-200 dark:border-emerald-900/60 bg-emerald-50/40 dark:bg-emerald-950/20'
                  : 'border-rose-200 dark:border-rose-900/60 bg-rose-50/40 dark:bg-rose-950/20'
              }`}
            >
              <div className="font-semibold text-slate-900 dark:text-white mb-2 flex items-start justify-between gap-2">
                <span>{idx + 1}. {q.question}</span>
                {isCorrect ? (
                  <span className="text-emerald-600 font-bold shrink-0">✓ Correct</span>
                ) : (
                  <span className="text-rose-600 font-bold shrink-0">✗ Missed</span>
                )}
              </div>
              <div className="text-slate-600 dark:text-slate-400 space-y-1">
                <div>Correct Answer: <strong className="text-emerald-700 dark:text-emerald-300">{q.options[q.correctIndex]}</stron
                {!isCorrect && userAns !== undefined && (
```

```
                    <div>Your Choice: <span className="text-rose-600 line-through">{q.options[userAns]}</span></div>
                  )}
                  <p className="mt-1 text-slate-500 dark:text-slate-400 italic">
                    Why: {q.explanation}
                  </p>
                </div>
              </div>
            );
          })}
        </div>
      </div>
    </div>
  )}

  {/* Quiz Past Performance History */}
  {!quizData && quizHistory.length > 0 && (
    <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 border border-slate-200/80 dark:border-slate-800/80 shadow-xs">
      <div className="flex items-center gap-2 text-slate-700 dark:text-slate-300 font-bold text-sm mb-4">
        <History className="w-4 h-4 text-purple-500" />
        <span>Recent Quiz Scores</span>
      </div>

      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-3">
        {quizHistory.slice(0, 6).map((rec) => (
          <div
            key={rec.id}
            className="p-3.5 rounded-2xl bg-slate-50 dark:bg-slate-800/50 border border-slate-200/60 dark:border-slate-700/60 flex i
          >
            <div>
              <div className="font-semibold text-slate-900 dark:text-white truncate max-w-[150px]">
                {rec.topic}
              </div>
              <div className="text-[10px] text-slate-400 mt-0.5">
                {rec.difficulty} • {rec.date}
              </div>
            </div>
            <div
              className={`font-black text-sm px-2.5 py-1 rounded-lg ${
                rec.percentage >= 70
                  ? 'bg-emerald-100 dark:bg-emerald-950/60 text-emerald-700 dark:text-emerald-300'
                  : 'bg-amber-100 dark:bg-amber-950/60 text-amber-700 dark:text-amber-300'
              }`}
            >
              {rec.percentage}%
            </div>
          </div>
        ))}
      </div>
    </div>
  )}
</div>
);
};
```

src/components/summarizer/NotesSummarizer.tsx

```
import React, { useState, useRef } from 'react';
import {
  FileText,
  Upload,
  Sparkles,
  FileCheck,
  X,
  Copy,
  Check,
  Download,
  List,
  Bookmark,
  AlignLeft,
} from 'lucide-react';
import { MarkdownRenderer } from '../common/MarkdownRenderer';
import { ErrorMessage } from '../common/ErrorMessage';

type FormatType = 'short_summary' | 'bullet_points' | 'key_terms';

const SAMPLE_NOTES = `Cellular respiration is a series of chemical reactions that break down glucose to produce ATP, which may be used a
Glycolysis produces 2 ATP molecules and occurs in the cytoplasm without oxygen (anaerobic).
The citric acid cycle takes place in the mitochondrial matrix and generates electron carriers (NADH and FADH2) along with 2 ATP.
Oxidative phosphorylation takes place across the inner mitochondrial membrane via the electron transport chain and ATP synthase, produci

export const NotesSummarizer: React.FC = () => {
  const [text, setText] = useState('');
  const [format, setFormat] = useState<FormatType>('short_summary');
  const [fileData, setFileData] = useState<{
    base64: string;
    mimeType: string;
    fileName: string;
    fileSizeKb: number;
  } | null>(null);

  const [isLoading, setIsLoading] = useState(false);
  const [summaryResult, setSummaryResult] = useState<string>('');
  const [errorState, setErrorState] = useState<{ message: string; isRateLimit: boolean } | null>(null);
  const [copied, setCopied] = useState(false);

  const fileInputRef = useRef<HTMLInputElement>(null);

  const handleFileUpload = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (!file) return;

    // Check size limit: max 10MB
    if (file.size > 10 * 1024 * 1024) {
      alert('File size exceeds 10MB limit. Please upload a smaller document.');
      return;
    }

    const reader = new FileReader();

    if (file.type === 'text/plain') {
      reader.onload = () => {
        const content = reader.result as string;
        setText(content);
        setFileData(null);
      };
      reader.readAsText(file);
    } else if (file.type === 'application/pdf') {
      reader.onload = () => {
        const result = reader.result as string;
        // Strip data:application/pdf;base64, prefix
        const base64 = result.split(',')[1];
        setFileData({
          base64,
          mimeType: file.type,
          fileName: file.name,
          fileSizeKb: Math.round(file.size / 1024),
        });
      };
      reader.readAsDataURL(file);
    } else {
      alert('Please upload a .txt or .pdf file.');
    }
  };

  const handleRemoveFile = () => {
    setFileData(null);
    if (fileInputRef.current) fileInputRef.current.value = '';
  };

  const handleSummarize = async () => {
    if (!text.trim() && !fileData) return;

    setIsLoading(true);
```

```
  setErrorState(null);

  try {
    const res = await fetch('/api/summarize', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        text: text.trim(),
        fileData,
        format,
      }),
    });

    const data = await res.json();
    if (!res.ok) {
      throw new Error(data.error || 'Failed to summarize notes.');
    }

    setSummaryResult(data.summary || '');
  } catch (err: any) {
    console.error('Summarize error:', err);
    const is429 = err.message?.includes('429') || err.message?.includes('Too many requests');
    setErrorState({
      message: err.message || 'Could not summarize notes. Please try again.',
      isRateLimit: is429,
    });
  } finally {
    setIsLoading(false);
  }
};

const handleCopy = () => {
  if (!summaryResult) return;
  navigator.clipboard.writeText(summaryResult);
  setCopied(true);
  setTimeout(() => setCopied(false), 2000);
};

const handleDownload = () => {
  if (!summaryResult) return;
  const blob = new Blob([summaryResult], { type: 'text/markdown;charset=utf-8' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = `edugenie-summary-${format}.md`;
  a.click();
  URL.revokeObjectURL(url);
};

const wordCount = text.trim() ? text.trim().split(/\s+/).length : 0;

return (
  <div className="max-w-4xl mx-auto space-y-6">
    {/* Input Card */}
    <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 sm:p-8 border border-slate-200/80 dark:border-slate-800/80 shadow-sm tr
      <div className="flex items-center gap-3 mb-5">
        <div className="p-3 rounded-2xl bg-indigo-100 dark:bg-indigo-950/60 text-indigo-600 dark:text-indigo-400">
          <FileText className="w-6 h-6" />
        </div>
        <div>
          <h2 className="text-xl font-bold text-slate-900 dark:text-white">
            Notes Summarizer
          </h2>
          <p className="text-sm text-slate-500 dark:text-slate-400">
            Paste lectures, book chapters, or upload a .txt/.pdf to distill the vital concepts.
          </p>
        </div>
      </div>

      {/* Format Selector Tabs */}
      <div className="mb-4">
        <label className="block text-xs font-semibold uppercase tracking-wider text-slate-500 dark:text-slate-400 mb-2">
          Select Output Format
        </label>
        <div className="grid grid-cols-1 sm:grid-cols-3 gap-2">
          {[
            { id: 'short_summary', label: 'Short Summary', icon: AlignLeft, desc: 'Executive overview & key takeaways' },
            { id: 'bullet_points', label: 'Bullet Points', icon: List, desc: 'Hierarchical study outline' },
            { id: 'key_terms', label: 'Key Terms & Defs', icon: Bookmark, desc: 'Vocabulary & formula glossary' },
          ].map((tab) => {
            const Icon = tab.icon;
            const isSelected = format === tab.id;
            return (
              <button
                key={tab.id}
                type="button"
                onClick={() => setFormat(tab.id as FormatType)}
                className={`p-3 rounded-2xl border text-left transition-all cursor-pointer ${
                  isSelected
```

```
              ? 'border-indigo-500 bg-indigo-50/70 dark:bg-indigo-950/50 text-indigo-900 dark:text-indigo-200 shadow-xs'
              : 'border-slate-200 dark:border-slate-800 hover:border-slate-300 dark:hover:border-slate-700 bg-slate-50/60 dark:b
          }`}
        >
          <div className="flex items-center gap-2 mb-1">
            <Icon className={`w-4 h-4 ${isSelected ? 'text-indigo-600 dark:text-indigo-400' : 'text-slate-500'}`} />
            <span className="font-bold text-xs">{tab.label}</span>
          </div>
          <p className="text-[11px] text-slate-500 dark:text-slate-400 leading-snug">{tab.desc}</p>
        </button>
      );
    })}
  </div>
</div>

{/* Input Text Area or Uploaded File Banner */}
<div className="space-y-3">
  {fileData ? (
    <div className="p-4 rounded-2xl bg-indigo-50/60 dark:bg-indigo-950/40 border border-indigo-200 dark:border-indigo-800/60 fle
      <div className="flex items-center gap-3">
        <div className="p-2.5 rounded-xl bg-indigo-600 text-white">
          <FileCheck className="w-5 h-5" />
        </div>
        <div>
          <div className="text-sm font-bold text-slate-900 dark:text-white truncate max-w-xs sm:max-w-md">
            {fileData.fileName}
          </div>
          <div className="text-xs text-indigo-600 dark:text-indigo-400">
            PDF Document • {fileData.fileSizeKb} KB ready for Gemini parsing
          </div>
        </div>
      </div>

      <button
        type="button"
        onClick={handleRemoveFile}
        className="p-1.5 rounded-lg text-slate-400 hover:text-rose-500 dark:hover:text-rose-400 transition-colors cursor-pointer
        title="Remove attached file"
      >
        <X className="w-5 h-5" />
      </button>
    </div>
  ) : (
    <div className="relative">
      <textarea
        value={text}
        onChange={(e) => setText(e.target.value)}
        rows={7}
        placeholder="Paste your study notes, essay, article, or lecture transcript here..."
        className="w-full px-4 py-3 rounded-2xl bg-slate-50 dark:bg-slate-800/70 border border-slate-200 dark:border-slate-700 t
      />
      <div className="flex items-center justify-between text-[11px] text-slate-400 px-1 mt-1">
        <span>{wordCount} words</span>
        {!text && (
          <button
            type="button"
            onClick={() => setText(SAMPLE_NOTES)}
            className="text-indigo-600 dark:text-indigo-400 hover:underline font-medium cursor-pointer"
          >
            Paste sample notes (Biology)
          </button>
        )}
      </div>
    </div>
  )}

  {/* Action Row: Upload File & Summarize Button */}
  <div className="flex flex-wrap items-center justify-between gap-3 pt-2">
    <div>
      <input
        ref={fileInputRef}
        type="file"
        accept=".txt,.pdf"
        onChange={handleFileUpload}
        className="hidden"
        id="file-upload"
      />
      <label
        htmlFor="file-upload"
        className="inline-flex items-center gap-2 px-3.5 py-2 rounded-xl border border-slate-200 dark:border-slate-700 bg-slate-
      >
        <Upload className="w-3.5 h-3.5" />
        <span>Upload .txt or .pdf file</span>
      </label>
    </div>

    <div className="flex items-center gap-2">
      {(text || fileData) && (
        <button
```

```
              type="button"
              onClick={() => {
                setText('');
                handleRemoveFile();
                setSummaryResult('');
              }}
              className="px-3 py-2 rounded-xl text-xs font-medium text-slate-500 hover:text-rose-500 dark:text-slate-400 transition-
            >
              Clear
            </button>
          )}

          <button
            type="button"
            onClick={handleSummarize}
            disabled={(!text.trim() && !fileData) || isLoading}
            className="px-5 py-2.5 rounded-xl bg-indigo-600 hover:bg-indigo-700 disabled:opacity-50 text-white font-semibold text-xs
          >
            {isLoading ? (
              <>
                <Sparkles className="w-3.5 h-3.5 animate-spin" />
                <span>Distilling Notes...</span>
              </>
            ) : (
              <>
                <Sparkles className="w-3.5 h-3.5" />
                <span>Generate Summary</span>
              </>
            )}
          </button>
        </div>
      </div>
    </div>

    {/* Error display */}
    {errorState && (
      <ErrorMessage
        message={errorState.message}
        isRateLimit={errorState.isRateLimit}
        onRetry={handleSummarize}
      />
    )}

    {/* Summary Output */}
    {summaryResult && (
      <div className="bg-white dark:bg-slate-900 rounded-3xl p-6 sm:p-8 border border-slate-200/80 dark:border-slate-800/80 shadow-sm
        <div className="flex flex-wrap items-center justify-between gap-3 pb-3 border-b border-slate-100 dark:border-slate-800">
          <div className="flex items-center gap-2">
            <span className="w-2.5 h-2.5 rounded-full bg-indigo-500"></span>
            <h3 className="font-bold text-base text-slate-900 dark:text-white capitalize">
              {format.replace('_', ' ')}
            </h3>
          </div>

          <div className="flex items-center gap-2">
            <button
              type="button"
              onClick={handleCopy}
              className="inline-flex items-center gap-1.5 px-3 py-1.5 rounded-xl border border-slate-200 dark:border-slate-700 text-xs
              title="Copy markdown"
            >
              {copied ? (
                <>
                  <Check className="w-3.5 h-3.5 text-emerald-500" />
                  <span className="text-emerald-600 dark:text-emerald-400">Copied</span>
                </>
              ) : (
                <>
                  <Copy className="w-3.5 h-3.5" />
                  <span>Copy</span>
                </>
              )}
            </button>

            <button
              type="button"
              onClick={handleDownload}
              className="inline-flex items-center gap-1.5 px-3 py-1.5 rounded-xl border border-slate-200 dark:border-slate-700 text-xs
              title="Download markdown file"
            >
              <Download className="w-3.5 h-3.5" />
              <span>Save .md</span>
            </button>
          </div>
        </div>

        <div className="pt-2">
          <MarkdownRenderer content={summaryResult} />
```

```
          </div>
        </div>
      )}
    </div>
  );
};
```

src/components/tutor/TutorChat.tsx

```
import React, { useState, useRef, useEffect } from 'react';
import {
  Send,
  Sparkles,
  Bot,
  User,
  Trash2,
  Square,
  BookOpen,
  HelpCircle,
  Lightbulb,
  GraduationCap,
  Copy,
  Check,
} from 'lucide-react';
import { ChatMessage } from '../../types';
import { MarkdownRenderer } from '../common/MarkdownRenderer';
import { ErrorMessage } from '../common/ErrorMessage';

interface TutorChatProps {
  messages: ChatMessage[];
  setMessages: React.Dispatch<React.SetStateAction<ChatMessage[]>>;
}

const STARTER_PROMPTS = [
  { label: 'Photosynthesis Analogy', prompt: 'Can you explain photosynthesis step-by-step using a kitchen or cooking analogy?' },
  { label: 'Calculus: Derivatives', prompt: 'What actually is a derivative in calculus? Explain with an intuitive real-life example.' },
  { label: 'Python: Recursion', prompt: 'How does recursion work in programming? Show a simple code example with step-by-step trace.' },
  { label: 'Newton’s 3rd Law', prompt: 'Explain Newton’s Third Law of Motion. Why does a rocket move upward in space if there is no air
];

export const TutorChat: React.FC<TutorChatProps> = ({ messages, setMessages }) => {
  const [input, setInput] = useState('');
  const [isStreaming, setIsStreaming] = useState(false);
  const [subject, setSubject] = useState('General');
  const [gradeLevel, setGradeLevel] = useState('High School / College');
  const [errorState, setErrorState] = useState<{ message: string; isRateLimit: boolean } | null>(null);
  const [copiedId, setCopiedId] = useState<string | null>(null);

  const messagesEndRef = useRef<HTMLDivElement>(null);
  const inputRef = useRef<HTMLTextAreaElement>(null);
  const abortControllerRef = useRef<AbortController | null>(null);

  // Auto-scroll to bottom of chat
  useEffect(() => {
    messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
  }, [messages, isStreaming]);

  const handleCopyMessage = (id: string, text: string) => {
    navigator.clipboard.writeText(text);
    setCopiedId(id);
    setTimeout(() => setCopiedId(null), 2000);
  };

  const handleClearHistory = () => {
    if (window.confirm('Clear current tutor conversation?')) {
      setMessages([]);
      setErrorState(null);
    }
  };

  const stopStreaming = () => {
    if (abortControllerRef.current) {
      abortControllerRef.current.abort();
      abortControllerRef.current = null;
    }
    setIsStreaming(false);
  };

  const handleSendMessage = async (customPrompt?: string) => {
    const textToSend = (customPrompt || input).trim();
    if (!textToSend || isStreaming) return;

    setErrorState(null);
    setInput('');

    // Append user message
    const userMessage: ChatMessage = {
      id: `user-${Date.now()}`,
      role: 'user',
      content: textToSend,
      timestamp: Date.now(),
    };

    const newMessages = [...messages, userMessage];
    setMessages(newMessages);
```

```
// Placeholder model message for streaming
const modelMessageId = `model-${Date.now()}`;
const modelPlaceholder: ChatMessage = {
  id: modelMessageId,
  role: 'model',
  content: '',
  timestamp: Date.now(),
};

setMessages([...newMessages, modelPlaceholder]);
setIsStreaming(true);

const controller = new AbortController();
abortControllerRef.current = controller;

try {
  // Prepare conversation history (last 10 messages for context efficiency)
  const recentHistory = newMessages.slice(-10).map((m) => ({
    role: m.role,
    content: m.content,
  }));

  const res = await fetch('/api/chat/stream', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      messages: recentHistory,
      subject: subject === 'General' ? '' : subject,
      gradeLevel,
    }),
    signal: controller.signal,
  });

  if (!res.ok) {
    const errorData = await res.json().catch(() => ({}));
    throw new Error(errorData.error || `Server responded with ${res.status}`);
  }

  const reader = res.body?.getReader();
  if (!reader) throw new Error('Streaming not supported by browser.');

  const decoder = new TextDecoder();
  let accumulatedText = '';

  while (true) {
    const { value, done } = await reader.read();
    if (done) break;

    const chunkStr = decoder.decode(value, { stream: true });
    const lines = chunkStr.split('\n');

    for (const line of lines) {
      if (line.startsWith('data: ')) {
        const dataStr = line.replace('data: ', '').trim();
        if (dataStr === '[DONE]') {
          break;
        }

        try {
          const parsed = JSON.parse(dataStr);
          if (parsed.error) {
            throw new Error(parsed.error);
          }
          if (parsed.text) {
            accumulatedText += parsed.text;
            // Update active model message content
            setMessages((prev) =>
              prev.map((msg) =>
                msg.id === modelMessageId ? { ...msg, content: accumulatedText } : msg
              )
            );
          }
        } catch (err: any) {
          if (err.message && !err.message.includes('JSON')) {
            throw err;
          }
        }
      }
    }
  }
} catch (err: any) {
  if (err.name === 'AbortError') {
    console.log('Stream generation stopped by user');
  } else {
    console.error('Chat error:', err);
    const errMsg = err?.message || 'Failed to communicate with AI Tutor.';
    const isRateLimit = errMsg.includes('Too many requests') || errMsg.includes('429');
    setErrorState({ message: errMsg, isRateLimit });

    // If the model message is empty, remove it
```

```
      setMessages((prev) =>
        prev.filter((msg) => !(msg.id === modelMessageId && !msg.content))
      );
    }
  } finally {
    setIsStreaming(false);
    abortControllerRef.current = null;
  }
};

const handleKeyDown = (e: React.KeyboardEvent<HTMLTextAreaElement>) => {
  if (e.key === 'Enter' && !e.shiftKey) {
    e.preventDefault();
    handleSendMessage();
  }
};

return (
  <div className="flex flex-col h-[calc(100vh-10rem)] max-w-5xl mx-auto w-full">
    {/* Settings Bar: Grade Level & Subject */}
    <div className="px-4 py-3 bg-white dark:bg-slate-900 rounded-2xl border border-slate-200/80 dark:border-slate-800/80 shadow-xs mb-
      <div className="flex flex-wrap items-center gap-2 sm:gap-4 text-xs">
        <div className="flex items-center gap-1.5 text-slate-500 dark:text-slate-400 font-medium">
          <GraduationCap className="w-4 h-4 text-indigo-500" />
          <span>Target Level:</span>
          <select
            value={gradeLevel}
            onChange={(e) => setGradeLevel(e.target.value)}
            className="bg-slate-100 dark:bg-slate-800 text-slate-800 dark:text-slate-200 font-semibold rounded-lg px-2.5 py-1 border b
          >
            <option value="Middle School">Middle School (Ages 11-14)</option>
            <option value="High School">High School (Ages 14-18)</option>
            <option value="High School / College">College / Undergraduate</option>
            <option value="Beginner / Simple">ELI5 (Simple terms)</option>
          </select>
        </div>

        <div className="flex items-center gap-1.5 text-slate-500 dark:text-slate-400 font-medium">
          <BookOpen className="w-4 h-4 text-sky-500" />
          <span>Subject:</span>
          <select
            value={subject}
            onChange={(e) => setSubject(e.target.value)}
            className="bg-slate-100 dark:bg-slate-800 text-slate-800 dark:text-slate-200 font-semibold rounded-lg px-2.5 py-1 border b
          >
            <option value="General">All / General</option>
            <option value="Mathematics">Mathematics</option>
            <option value="Physics">Physics</option>
            <option value="Chemistry">Chemistry</option>
            <option value="Biology">Biology</option>
            <option value="Computer Science">Computer Science</option>
            <option value="History">History</option>
            <option value="Literature">Literature</option>
          </select>
        </div>
      </div>

      {messages.length > 0 && (
        <button
          type="button"
          onClick={handleClearHistory}
          className="flex items-center gap-1 text-xs font-medium text-slate-500 hover:text-rose-500 dark:text-slate-400 dark:hover:tex
          title="Clear Chat Messages"
        >
          <Trash2 className="w-3.5 h-3.5" />
          <span className="hidden sm:inline">Clear Chat</span>
        </button>
      )}
    </div>

    {/* Messages Scroll Area */}
    <div className="flex-1 overflow-y-auto px-1 sm:px-2 space-y-4 pb-4">
      {messages.length === 0 ? (
        <div className="h-full flex flex-col items-center justify-center text-center p-6 sm:p-10 my-auto">
          <div className="w-16 h-16 rounded-3xl bg-gradient-to-tr from-indigo-500 via-indigo-600 to-sky-400 flex items-center justify-
            <Bot className="w-8 h-8" />
          </div>
          <h2 className="text-xl sm:text-2xl font-bold text-slate-900 dark:text-white tracking-tight">
            Hello! I'm your AI Learning Tutor
          </h2>
          <p className="text-sm text-slate-500 dark:text-slate-400 max-w-md mt-2 leading-relaxed">
            Ask me any doubt or problem. I'll explain step-by-step with analogies and examples, then check if it makes sense!
          </p>

          {/* Quick Prompts */}
          <div className="mt-8 w-full max-w-xl">
            <div className="text-xs font-semibold uppercase tracking-wider text-slate-400 dark:text-slate-500 mb-3 flex items-center j
              <Lightbulb className="w-3.5 h-3.5 text-amber-500" />
              <span>Try an example question</span>
```

```
        </div>
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-2.5">
          {STARTER_PROMPTS.map((sp, idx) => (
            <button
              key={idx}
              type="button"
              onClick={() => handleSendMessage(sp.prompt)}
              className="p-3 text-left rounded-xl bg-white dark:bg-slate-900 border border-slate-200/90 dark:border-slate-800 hove
            >
              <div className="font-semibold text-slate-800 dark:text-slate-200 group-hover:text-indigo-600 dark:group-hover:text-i
                {sp.label}
              </div>
              <div className="text-slate-500 dark:text-slate-400 line-clamp-1 mt-0.5">
                {sp.prompt}
              </div>
            </button>
          ))}
        </div>
      </div>
    </div>
  ) : (
    messages.map((message) => {
      const isUser = message.role === 'user';
      return (
        <div
          key={message.id}
          className={`flex gap-3 sm:gap-4 ${isUser ? 'justify-end' : 'justify-start'}`}
        >
          {!isUser && (
            <div className="w-8 h-8 rounded-xl bg-gradient-to-tr from-indigo-600 to-sky-500 flex items-center justify-center text-
              <Bot className="w-4 h-4" />
            </div>
          )}

          <div
            className={`relative group max-w-[88%] sm:max-w-[80%] rounded-2xl p-4 sm:p-5 shadow-xs transition-all ${
              isUser
                ? 'bg-indigo-600 text-white rounded-tr-xs'
                : 'bg-white dark:bg-slate-900 text-slate-800 dark:text-slate-100 border border-slate-200/80 dark:border-slate-800/
            }`}
          >
            {/* Copy button */}
            <button
              type="button"
              onClick={() => handleCopyMessage(message.id, message.content)}
              className={`absolute top-2 right-2 p-1.5 rounded-lg text-xs opacity-0 group-hover:opacity-100 transition-opacity cur
                isUser
                  ? 'text-indigo-200 hover:text-white hover:bg-indigo-700/60'
                  : 'text-slate-400 hover:text-slate-600 dark:hover:text-slate-200 hover:bg-slate-100 dark:hover:bg-slate-800'
              }`}
              title="Copy message"
            >
              {copiedId === message.id ? (
                <Check className="w-3.5 h-3.5 text-emerald-400" />
              ) : (
                <Copy className="w-3.5 h-3.5" />
              )}
            </button>

            {isUser ? (
              <div className="whitespace-pre-wrap text-sm sm:text-base leading-relaxed">
                {message.content}
              </div>
            ) : message.content ? (
              <MarkdownRenderer content={message.content} />
            ) : (
              <div className="flex items-center gap-2 text-indigo-600 dark:text-indigo-400 py-1">
                <Sparkles className="w-4 h-4 animate-spin" />
                <span className="text-xs font-semibold">Tutor is thinking step-by-step...</span>
              </div>
            )}

            <div
              className={`text-[10px] mt-2 flex items-center justify-end ${
                isUser ? 'text-indigo-200' : 'text-slate-400 dark:text-slate-500'
              }`}
            >
              {new Date(message.timestamp).toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })}
            </div>
          </div>

          {isUser && (
            <div className="w-8 h-8 rounded-xl bg-slate-200 dark:bg-slate-800 flex items-center justify-center text-slate-700 dark
              <User className="w-4 h-4" />
            </div>
          )}
        </div>
      );
    })
```

```
        )}

        <div ref={messagesEndRef} />
      </div>

      {/* Error banner if present */}
      {errorState && (
        <ErrorMessage
          message={errorState.message}
          isRateLimit={errorState.isRateLimit}
          onRetry={() => {
            const lastUser = [...messages].reverse().find((m) => m.role === 'user');
            if (lastUser) handleSendMessage(lastUser.content);
          }}
        />
      )}

      {/* Chat Input Container */}
      <div className="bg-white dark:bg-slate-900 rounded-2xl border border-slate-200/90 dark:border-slate-800 shadow-md p-2 sm:p-2.5">
        <form
          onSubmit={(e) => {
            e.preventDefault();
            handleSendMessage();
          }}
          className="flex items-end gap-2"
        >
          <textarea
            ref={inputRef}
            value={input}
            onChange={(e) => setInput(e.target.value)}
            onKeyDown={handleKeyDown}
            placeholder="Type your question or homework doubt... (Press Enter to send, Shift+Enter for newline)"
            rows={2}
            disabled={isStreaming}
            className="flex-1 resize-none bg-transparent px-3 py-2 text-sm text-slate-900 dark:text-white placeholder:text-slate-400 foc
          />

          <div className="flex items-center gap-1.5 pb-1">
            {isStreaming ? (
              <button
                type="button"
                onClick={stopStreaming}
                className="p-2.5 rounded-xl bg-rose-500 hover:bg-rose-600 text-white transition-all shadow-sm cursor-pointer"
                title="Stop response"
              >
                <Square className="w-4 h-4 fill-white" />
              </button>
            ) : (
              <button
                type="submit"
                disabled={!input.trim()}
                className="p-2.5 rounded-xl bg-indigo-600 hover:bg-indigo-700 disabled:opacity-40 disabled:hover:bg-indigo-600 text-whit
                title="Send message"
              >
                <Send className="w-4 h-4" />
              </button>
            )}
          </div>
        </form>
      </div>
    </div>
  );
};
```

src/index.css

```
@import "tailwindcss";

@layer base {
  body {
    font-family: 'Plus Jakarta Sans', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  }
  code, pre, kbd {
    font-family: 'JetBrains Mono', monospace;
  }
}

/* 3D Flip Card Utilities */
.perspective-1000 {
  perspective: 1000px;
}
.transform-style-3d {
  transform-style: preserve-3d;
}
.backface-hidden {
  backface-visibility: hidden;
  -webkit-backface-visibility: hidden;
}
.rotate-y-180 {
  transform: rotateY(180deg);
}

/* Custom subtle scrollbar */
::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}
::-webkit-scrollbar-track {
  background: transparent;
}
::-webkit-scrollbar-thumb {
  background: rgba(156, 163, 175, 0.4);
  border-radius: 9999px;
}
::-webkit-scrollbar-thumb:hover {
  background: rgba(156, 163, 175, 0.7);
}
```

src/main.tsx

```
import {createRoot} from 'react-dom/client';
import App from './App.tsx';
import './index.css';

createRoot(document.getElementById('root')!).render(<App />);
```

src/types/index.ts

```
export type TabType = 'tutor' | 'explainer' | 'summarizer' | 'quiz' | 'flashcards' | 'planner';

export interface ChatMessage {
  id: string;
  role: 'user' | 'model';
  content: string;
  timestamp: number;
}

export interface ConceptExplanation {
  topic: string;
  level: 'Beginner' | 'Intermediate' | 'Advanced';
  simpleDefinition: string;
  analogy: string;
  keyPoints: string[];
  realWorldExample: string;
  recap: string;
  quickCheck: {
    question: string;
    options: string[];
    correctIndex: number;
    explanation: string;
  };
}

export interface QuizQuestion {
  id: number;
  question: string;
  options: string[];
  correctIndex: number;
  explanation: string;
}

export interface QuizData {
  topic: string;
  difficulty: 'Easy' | 'Medium' | 'Hard';
  questions: QuizQuestion[];
}

export interface QuizResultRecord {
  id: string;
  topic: string;
  score: number;
  total: number;
  percentage: number;
  date: string;
  difficulty: string;
}

export interface Flashcard {
  id: string;
  front: string;
  back: string;
  hint: string;
  category: string;
  known?: boolean;
}

export interface FlashcardDeck {
  deckTitle: string;
  cards: Flashcard[];
}

export interface StudyBlock {
  timeSlot: string;
  subject: string;
  task: string;
  technique: string;
  completed?: boolean;
}

export interface StudyDay {
  dayNumber: number;
  dateOrLabel: string;
  theme: string;
  isRevisionDay: boolean;
  blocks: StudyBlock[];
  dailyTip: string;
}

export interface StudyPlan {
  planTitle: string;
  overview: string;
  totalDays: number;
  dailyHoursTarget: number;
  days: StudyDay[];
  createdDate: string;
```

```
}
```

src/utils/storage.ts

```
import { ChatMessage, QuizResultRecord, Flashcard, StudyPlan } from '../types';

const STORAGE_KEYS = {
  THEME: 'edugenie_theme',
  CHAT_MESSAGES: 'edugenie_chat_messages',
  QUIZ_HISTORY: 'edugenie_quiz_history',
  FLASHCARDS: 'edugenie_flashcards',
  STUDY_PLAN: 'edugenie_study_plan',
  SAVED_CONCEPT: 'edugenie_saved_concept',
};

// Safe JSON parse wrapper
function safeGet<T>(key: string, defaultValue: T): T {
  try {
    const item = localStorage.getItem(key);
    return item ? JSON.parse(item) : defaultValue;
  } catch (e) {
    console.warn(`Error reading localStorage key "${key}":`, e);
    return defaultValue;
  }
}

function safeSet<T>(key: string, value: T): void {
  try {
    localStorage.setItem(key, JSON.stringify(value));
  } catch (e) {
    console.warn(`Error writing to localStorage key "${key}":`, e);
  }
}

// Chat storage
export const getStoredChat = (): ChatMessage[] => safeGet<ChatMessage[]>(STORAGE_KEYS.CHAT_MESSAGES, []);
export const saveStoredChat = (messages: ChatMessage[]) => safeSet(STORAGE_KEYS.CHAT_MESSAGES, messages);
export const clearStoredChat = () => localStorage.removeItem(STORAGE_KEYS.CHAT_MESSAGES);

// Quiz history storage
export const getStoredQuizHistory = (): QuizResultRecord[] => safeGet<QuizResultRecord[]>(STORAGE_KEYS.QUIZ_HISTORY, []);
export const saveQuizResult = (record: QuizResultRecord) => {
  const current = getStoredQuizHistory();
  const updated = [record, ...current].slice(0, 30); // keep last 30 quizzes
  safeSet(STORAGE_KEYS.QUIZ_HISTORY, updated);
};
export const clearStoredQuizzes = () => localStorage.removeItem(STORAGE_KEYS.QUIZ_HISTORY);

// Flashcards storage
export const getStoredFlashcards = (): { deckTitle: string; cards: Flashcard[] } | null =>
  safeGet<{ deckTitle: string; cards: Flashcard[] } | null>(STORAGE_KEYS.FLASHCARDS, null);
export const saveStoredFlashcards = (deck: { deckTitle: string; cards: Flashcard[] }) =>
  safeSet(STORAGE_KEYS.FLASHCARDS, deck);
export const clearStoredFlashcards = () => localStorage.removeItem(STORAGE_KEYS.FLASHCARDS);

// Study Plan storage
export const getStoredStudyPlan = (): StudyPlan | null => safeGet<StudyPlan | null>(STORAGE_KEYS.STUDY_PLAN, null);
export const saveStoredStudyPlan = (plan: StudyPlan) => safeSet(STORAGE_KEYS.STUDY_PLAN, plan);
export const clearStoredStudyPlan = () => localStorage.removeItem(STORAGE_KEYS.STUDY_PLAN);

// Concept explanation cache
export const getStoredConcept = (): any => safeGet(STORAGE_KEYS.SAVED_CONCEPT, null);
export const saveStoredConcept = (concept: any) => safeSet(STORAGE_KEYS.SAVED_CONCEPT, concept);

// Theme storage ('light' | 'dark')
export const getStoredTheme = (): 'light' | 'dark' => {
  try {
    const saved = localStorage.getItem(STORAGE_KEYS.THEME);
    if (saved === 'dark' || saved === 'light') return saved;
    return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light';
  } catch {
    return 'light';
  }
};
export const saveStoredTheme = (theme: 'light' | 'dark') => {
  try {
    localStorage.setItem(STORAGE_KEYS.THEME, theme);
  } catch {}
};

// Clear All Session data
export const clearAllUserData = () => {
  try {
    localStorage.removeItem(STORAGE_KEYS.CHAT_MESSAGES);
    localStorage.removeItem(STORAGE_KEYS.QUIZ_HISTORY);
    localStorage.removeItem(STORAGE_KEYS.FLASHCARDS);
    localStorage.removeItem(STORAGE_KEYS.STUDY_PLAN);
    localStorage.removeItem(STORAGE_KEYS.SAVED_CONCEPT);
  } catch (e) {
    console.error('Failed to clear session data:', e);
  }
```

```
};
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "experimentalDecorators": true,
    "useDefineForClassFields": false,
    "module": "ESNext",
    "types": ["vite/client"],
    "lib": [
      "ES2022",
      "DOM",
      "DOM.Iterable"
    ],
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "isolatedModules": true,
    "moduleDetection": "force",
    "allowJs": true,
    "jsx": "react-jsx",
    "paths": {
      "@/*": [
        "./*"
      ]
    },
    "allowImportingTsExtensions": true,
    "noEmit": true
  }
}
```

vite.config.ts

```
import tailwindcss from '@tailwindcss/vite';
import react from '@vitejs/plugin-react';
import path from 'path';
import {defineConfig} from 'vite';

export default defineConfig(() => {
  return {
    plugins: [react(), tailwindcss()],
    resolve: {
      alias: {
        '@': path.resolve(__dirname, '.'),
      },
    },
    server: {
      // HMR is disabled in AI Studio via DISABLE_HMR env var.
      // Do not modify—file watching is disabled to prevent flickering during agent edits.
      hmr: process.env.DISABLE_HMR !== 'true',
      // Disable file watching when DISABLE_HMR is true to save CPU during agent edits.
      watch: process.env.DISABLE_HMR === 'true' ? null : {},
    },
  };
});
```